Automotive ECU Firmware

Reverse Engineering

Chapter I: Documentation & Fundamentals

Chapter II: Loading your Binary

Chapter III: Reverse Engineering

Chapter IV: Testing and Validation

SAAD Ahmad

June 1, 2026

Contents

About the Lab Binary

This laboratory uses `synthetic_tc1766.bin`, a purpose-built firmware image that replicates the memory layout, address map, and algorithm structure of a real TriCore TC1766 ECU. The binary was generated from scratch and contains no proprietary manufacturer code. All addresses, function locations, and algorithm structures documented in this report are valid for the binary. Polynomial constants and hardware magic numbers differ intentionally from any production firmware.

I Documentation & Fundamentals

I.1 The ECU: Architecture & Ecosystem

An Electronic Control Unit (ECU) is a specialized embedded computer responsible for controlling one or more electrical subsystems of a vehicle. A modern automobile typically contains between 50 and 150 ECUs, ranging from the engine control module (ECM) that manages combustion, to simple body control modules (BCM) that handle window motors and courtesy lights.

From a reverse engineer's perspective, an ECU is a fixed-function embedded system: it has no operating system shell, no user-accessible filesystem, and no network stack in the traditional IT sense. Its entire behavior is encoded in firmware burned into Flash memory.

Figure I.1: Automotive ECU

I.1.1 Taxonomy of ECUs

The table below classifies the primary ECU types found in a modern vehicle. The two rows highlighted in yellow the **Engine Control Module (ECM/EMS)** and the **Central Gateway (CGW)** represent the highest-value targets for a security researcher: the ECM because it controls safety-critical outputs, and the CGW because it bridges all vehicle sub-networks.

Table I.1: ECU Taxonomy in Modern Vehicles

Domain	Unit Name	Key Functions	Typical Bus
Powertrain	**Engine Control Module (ECM/EMS)**	Fuel injection, ignition, emissions	CAN-HS
Powertrain	Transmission Control Unit (TCU)	Gear selection, torque management	CAN-HS
Chassis	Electronic Stability Program (ESP/ESC)	ABS, traction, yaw control	CAN-HS
Body	Body Control Module (BCM)	Lighting, locks, wipers, windows	CAN-LS / LIN
Safety	Airbag Control Unit (ACU)	Crash detection, pyro activation	CAN-HS
ADAS	Advanced Driver Assist ECU	Camera/radar fusion, lane keeping	Ethernet / FlexRay
Telematics	Telematic Control Unit (TCU)	Cellular/GPS connectivity	Ethernet + CAN
Diagnostics	**Central Gateway (CGW)**	Protocol bridging, firewall	All buses

I.2 Vehicle Network Architecture

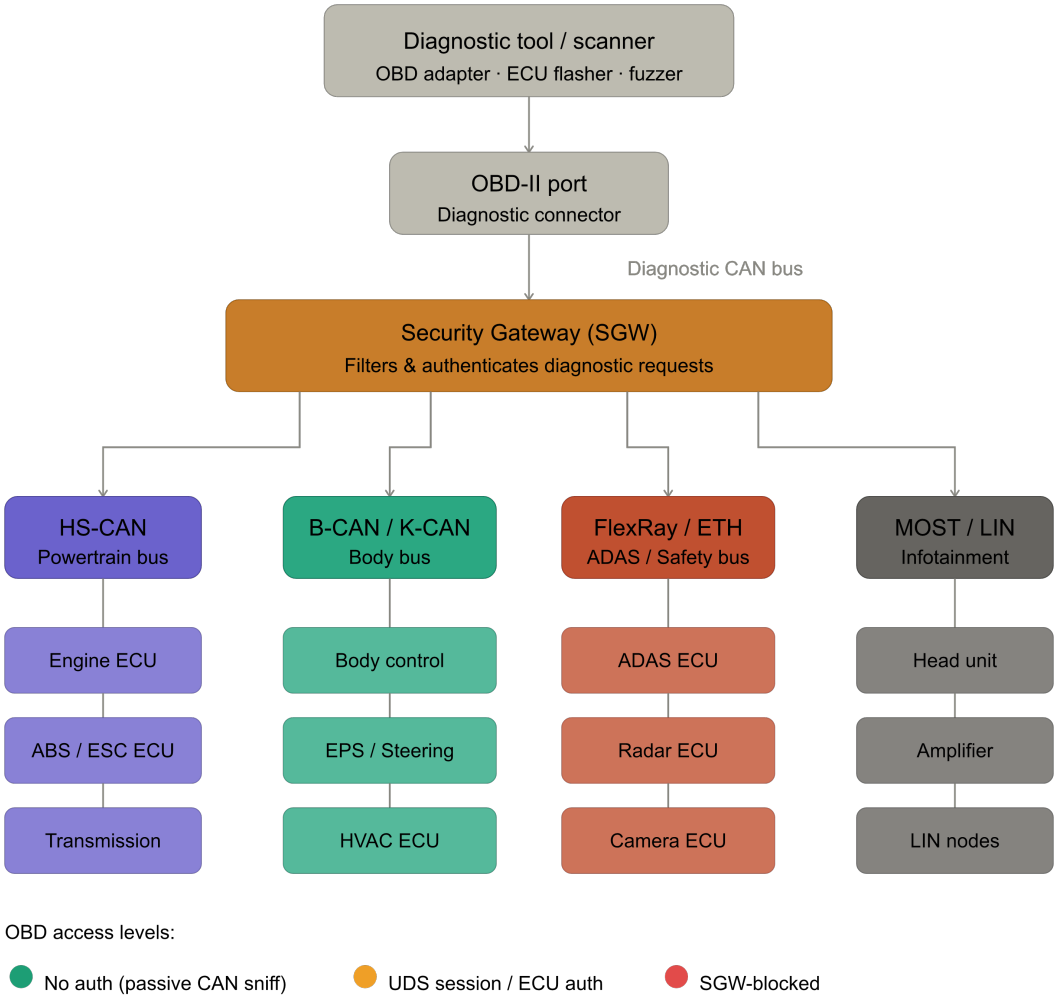


Figure I.2: Vehicle ECU Network Topology with OBD-II Access Point and Secure Gateway

I.2.1 The ECU is Not Alone

A vehicle's ECUs communicate over shared serial buses. Understanding this topology is critical for two reasons:

- **Entry points:** Which bus is physically reachable from outside the vehicle?
- **Lateral movement:** Once on one bus, which ECUs can you reach?

I.2.2 Bus Systems

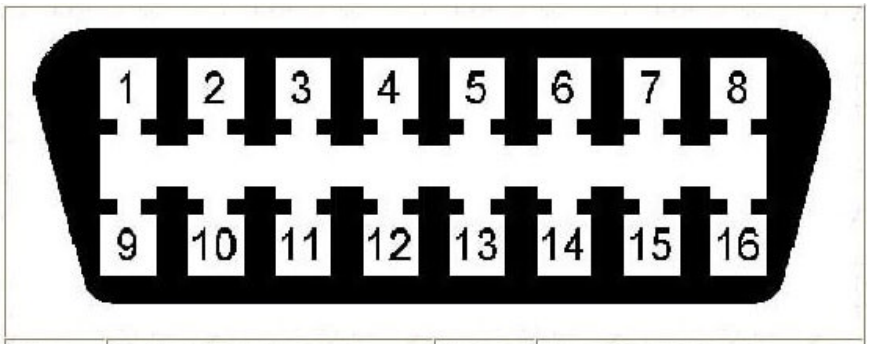
CAN Bus (ISO 11898) : The Backbone

The Controller Area Network is the dominant bus in automotive systems. Developed by Bosch in the 1980s, it is a differential two-wire bus (`CAN_H` / `CAN_L`) with multi-master, broadcast messaging and built-in error detection.

Figure I.3: CAN Bus Differential Signalling and Frame Structure

I.2.3 The OBD-II Port: Your Physical Interface

The On-Board Diagnostics II (OBD-II) port (SAE J1962) is a 16-pin trapezoid connector mandated by law in all passenger vehicles. It is your primary physical access point for firmware extraction.



PIN	DESCRIPTION	PIN	DESCRIPTION
1	Vendor Option	9	Vendor Option
2	J1850 Bus +	10	j1850 BUS
3	Vendor Option	11	Vendor Option
4	Chassis Ground	12	Vendor Option
5	Signal Ground	13	Vendor Option
6	CAN (J-2234) High	14	CAN (J-2234) Low
7	ISO 9141-2 K-Line	15	ISO 9141-2 Low
8	Vendor Option	16	Battery Power

OBD-II Connector and Pinout

Figure I.4: OBD-II Connector Pin Layout (SAE J1962) : Pins 6 and 14 expose CAN-HS

Pins 6 and 14 give you direct access to the high-speed powertrain CAN bus. A low-cost USB-to-CAN adapter is sufficient to start sending diagnostic frames.

I.2.4 Gateway ECUs: The Network Backbone

The Central Gateway (CGW) is the most security-critical ECU in a modern vehicle. It:

- Bridges isolated sub-networks (prevents a compromised infotainment ECU from directly messaging the braking ECU).
- Routes diagnostic messages from the OBD-II port to the correct domain ECU.
- Filters messages based on allowlists (or fails to, creating exploitable vulnerabilities).

When you plug into OBD-II, you are physically connecting to one specific CAN bus (typically the powertrain bus). The gateway decides what else you can reach. Understanding gateway routing tables is therefore a core skill in automotive penetration testing.

> **Note:** Part 2 of this guide does not cover Secure Gateway (SGW) bypass. This typically requires a digital certificate or a manufacturer-signed token, often obtained via an internet connection to the OEM's servers. Throughout the practical exercises we assume the gateway passes diagnostic messages through to the target ECUs.

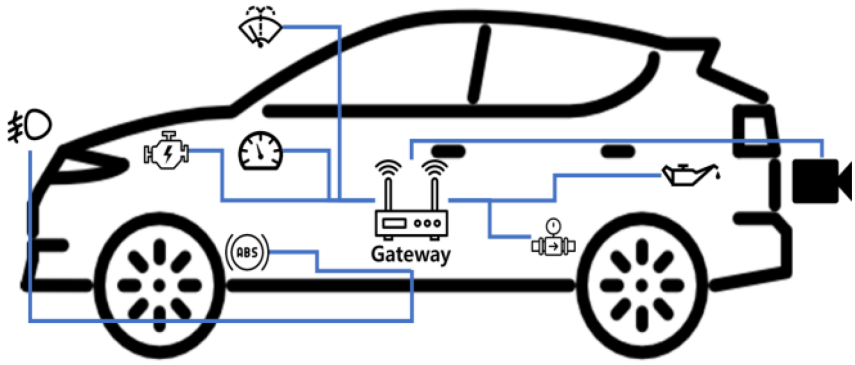


Figure I.5: Central Gateway (CGW) or Secure Gateway (SGW)

I.3 Communication Protocols

I.3.1 UDS: Unified Diagnostic Services (ISO 14229)

UDS is the industry-standard protocol for ECU diagnostics, flashing, and security operations. It defines a set of services, each identified by a one-byte **Service Identifier (SID)**. Understanding UDS is the single most important skill for automotive firmware extraction.

UDS Session Model

The ECU operates in one of several diagnostic sessions, each unlocking a different set of capabilities:

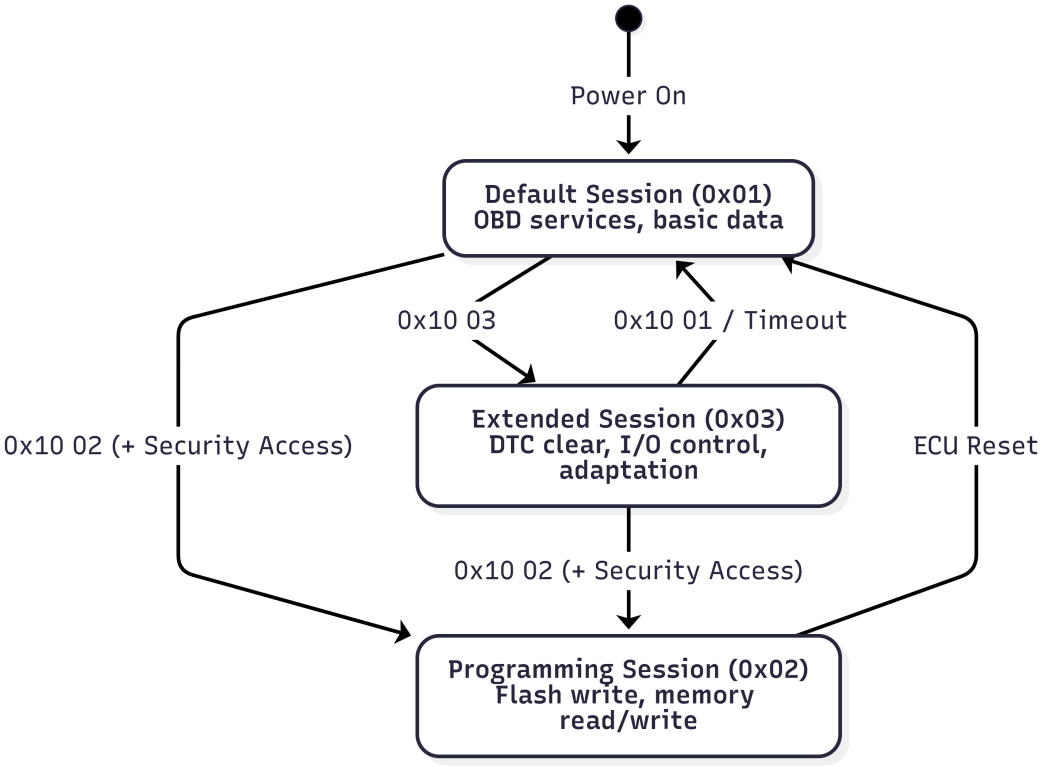


Figure I.6: UDS Diagnostic Session Model and Allowed Transitions

Critical UDS Services for Firmware Reverse Engineering

Table I.2: Critical UDS Services for Firmware Reverse Engineering

SID (Req)	SID (Resp)	Service Name	RE Relevance
`0x10`	`0x50`	DiagnosticSessionControl	Enter programming/extended session
`0x11`	`0x51`	ECUReset	Restart the ECU (hard/soft/key-off)
`0x14`	`0x54`	ClearDiagnosticInformation	Erase DTCs
`0x19`	`0x59`	ReadDTCInformation	List fault codes
`0x22`	`0x62`	ReadDataByIdentifier	Read VIN, calibration IDs, fingerprints
`0x23`	`0x63`	ReadMemoryByAddress	Direct memory dump (if not blocked)
`0x27`	`0x67`	SecurityAccess	Seed-Key authentication (see Section I.6.2)
`0x2E`	`0x6E`	WriteDataByIdentifier	Write VIN, adaptation values
`0x2F`	`0x6F`	InputOutputControlByIdentifier	Force outputs (open actuators)
`0x31`	`0x71`	RoutineControl	Run flash erase, CRC verify routines
`0x34`	`0x74`	RequestDownload	Initiate firmware upload to ECU
`0x36`	`0x76`	TransferData	Send firmware data blocks
`0x37`	`0x77`	RequestTransferExit	Finalize firmware transfer
`0x3E`	`0x7E`	TesterPresent	Keep session alive (heartbeat)

Negative Response Codes (NRC)

When the ECU rejects a request it returns a Negative Response Code (NRC). The table below lists the codes most commonly encountered during firmware extraction attempts.

Table I.3: Common UDS Negative Response Codes and Their Interpretation

Code	Meaning	RE Interpretation
`0x22`	conditionsNotCorrect	Wrong session : switch session first
`0x31`	requestOutOfRange	Address or identifier not supported
`0x33`	securityAccessDenied	Security Access not yet performed
`0x35`	invalidKey	Wrong seed-key response
`0x24`	requestSequenceError	Steps executed out of order

Typical Firmware Extraction Sequence

The following message exchange illustrates a complete UDS firmware extraction flow, assuming the ECU permits `ReadMemoryByAddress` after authentication.

Table I.4: UDS Firmware Extraction Message Sequence (Successful Case)

Dir.	Hex Data	Description
→	`10 02`	Enter Programming Session
←	`50 02 ...`	Positive Response
→	`27 01`	Request Seed (Level 01)
←	`67 01 67 01 DE AD BE EF`	Seed returned (`0xDEADBEEF`)
→	`27 02 [4 bytes]`	Send computed Key
←	`67 02`	Security Access Granted
→	`23 12 A0000000 00800000`	ReadMemoryByAddress (Addr: `0xA0000000`, Len: `0x800000`)
←	`63 [data...]`	Flash data returned

Important: Most production ECUs block `ReadMemoryByAddress` (`0x23`) even after security access is granted. Real firmware extraction usually involves exploiting an undocumented vendor routine, using a calibration or XCP interface, or performing a hardware JTAG attack.

I.4 Inside the Box

I.4.1 Anatomy of an Electronic Control Unit

An Electronic Control Unit is essentially a specialized computer purpose-built for a vehicle. While the MCU (the chip, such as the Infineon TC1766) acts as the central processing brain, the ECU is the complete assembly that hosts and supports it. The diagram below illustrates how the internal functional blocks are organized to handle everything from fuel injection to high-speed data networking.

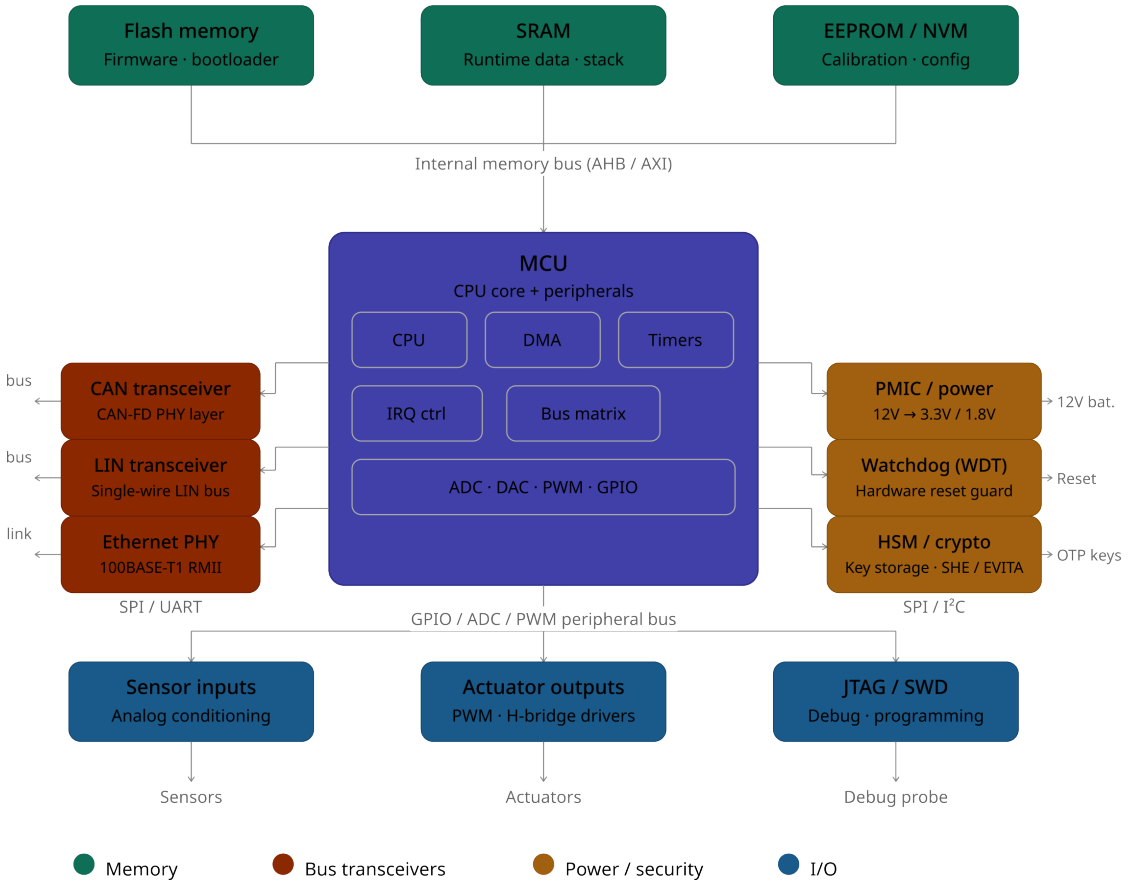


Figure I.7: Internal Architecture of an ECU Showing the MCU, Memory Cluster, and Peripheral Interfaces

The **Memory Cluster** (top of the diagram) is the most critical area for understanding ECU behavior: this is where the firmware resides and from where the CPU fetches instructions. The following sections examine the two dominant storage types Internal Flash and EEPROM and identify the memory segments most relevant to diagnostic communication and calibration data.

I.4.2 Architectural Address Map (TC1766)

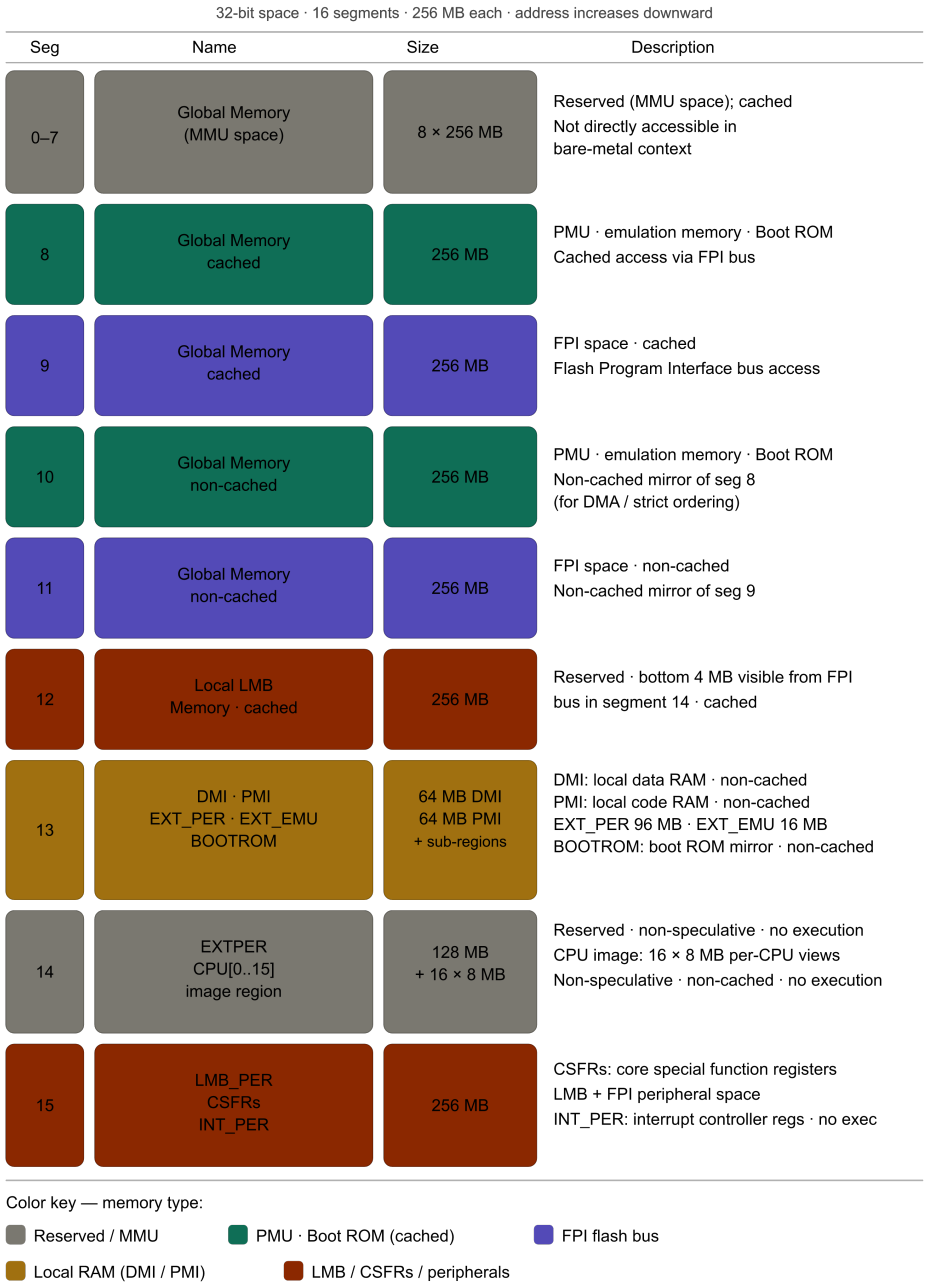


Figure I.8: TC1766 32-Bit Memory Address Map (16 Segments of 256 MB Each)

The TC1766 address space is divided into 16 fixed 256 MB segments primarily to simplify hardware decoding and to allow the assignment of distinct memory attributes such as caching policy, speculative execution permission, or peripheral access to each range.

The table below summarizes the five segments of greatest importance during firmware analysis:

Table I.5: TC1766 Address Segments of Primary Relevance for Firmware Analysis

Segment	Base Address	Description	Primary Use Case
8	`0x8000 0000`	Program Flash (Cached)	Analyzing main application logic and diagnostic code
10	`0xA000 0000`	Program Flash (Non-cached)	Tracing initial boot instructions and the reset entry point
13 (Code)	`0xC000 0000`	Scratch-Pad RAM (SPRAM)	Analyzing code relocated to RAM for high-speed security math
13 (Data)	`0xD000 0000`	Local Data RAM (LDRAM)	Monitoring live stack, runtime variables, and generated seeds/keys
15	`0xF000 0000`	Peripherals (CSFRs/SFRs)	Accessing CAN hardware registers for incoming diagnostic requests

I.4.3 The Reset Vector and Interrupt Vector Table

When the TC1766 powers on or is reset, the CPU always fetches its first instruction from `0xA000_0000`. This behavior is hardwired and cannot be altered by software. On TriCore, the trap and interrupt base address is configured by the **BTV** (Base Trap Vector) register, and the reset handler begins at the start of Program Flash.

The Interrupt Vector Table (IVT) on TriCore differs structurally from ARM-based architectures: it is not a flat table of addresses but a table of *code blocks*. Each entry is 8 words (32 bytes) wide and contains handler code directly, rather than a pointer to it. The BTV register points to the base of this table.

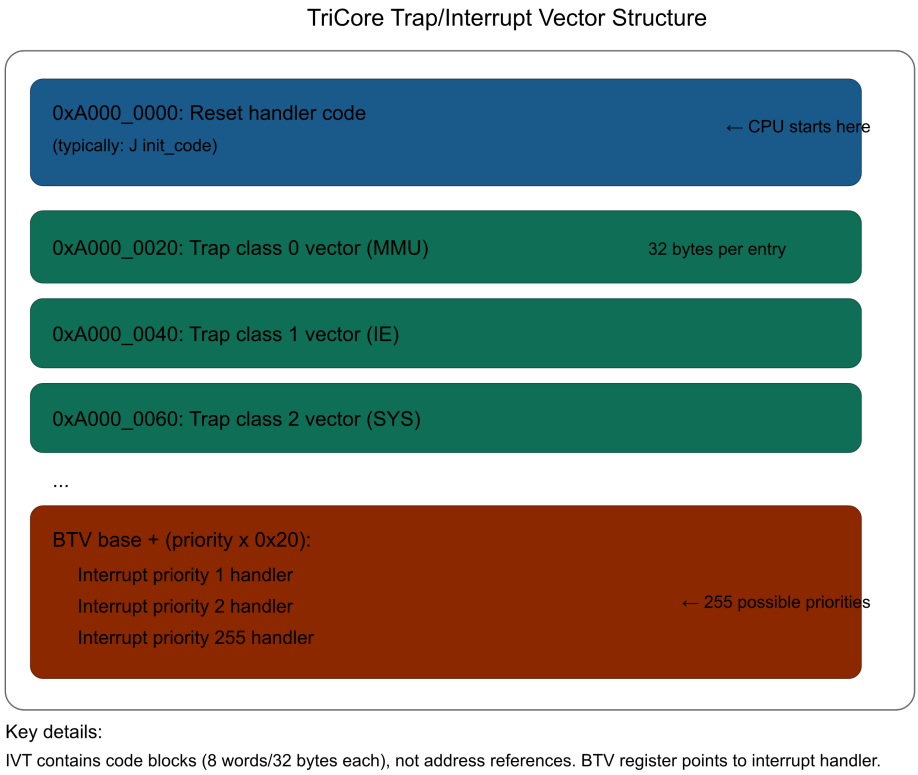


Figure I.9: TriCore Trap and Interrupt Vector Table Structure

I.4.4 Flash vs. EEPROM

Property	Program Flash (PFlash)	Data Flash (DFlash)
TC1766 Size	1504 KB	32 KB
Base Address	0xA000_0000	0xA800_0000
Contains	Code, constants, calibration	NVM adaptations, DTCs
Erase Granularity	Sectors (≈16 KB)	Sectors (≈1 KB)
Write Endurance	≈10,000 cycles	≈100,000 to 500,000 cycles
RE Value	High: contains all logic	Medium: contains config data

Figure I.10: Side-by-Side Comparison of Flash and EEPROM Memory Characteristics

I.5 Firmware File Formats

Several file formats appear throughout the automotive firmware toolchain:

- **Intel HEX** (`.hex`)
- **Motorola S-Record** (`.s19` / `.srec` / `.mot`)
- **Binary** (`.bin`) : the raw format
- **A2L Files (ASAP2)** : the reverse engineer's measurement and calibration reference database

Each format carries its own trade-offs. This guide covers only the raw binary format, as it is the format produced directly by hardware extraction techniques.

Binary (`.bin`) : The Raw Format

A raw binary is a byte-for-byte copy of memory. It contains no header and no embedded address information: only raw data. When firmware is extracted via JTAG, a hardware flash programmer, or a successful UDS `ReadMemoryByAddress` call, the output is typically a `.bin` file.

> **Important: Critical RE challenge:** You must know the correct base address before loading the binary into a disassembler. Loading a TC1766 firmware image at offset `0x0` will produce completely incorrect results for every absolute address reference. Base address determination is examined in detail in the practical section.

I.6 Security Mechanisms

I.6.1 Checksums and CRCs: Integrity Verification

CRC-32 (Most Common)

A 32-bit Cyclic Redundancy Check is computed over a defined memory range (typically all of Program Flash except the CRC word itself). The resulting value is stored at a known location, most often the last four bytes of flash or at a fixed linker symbol.

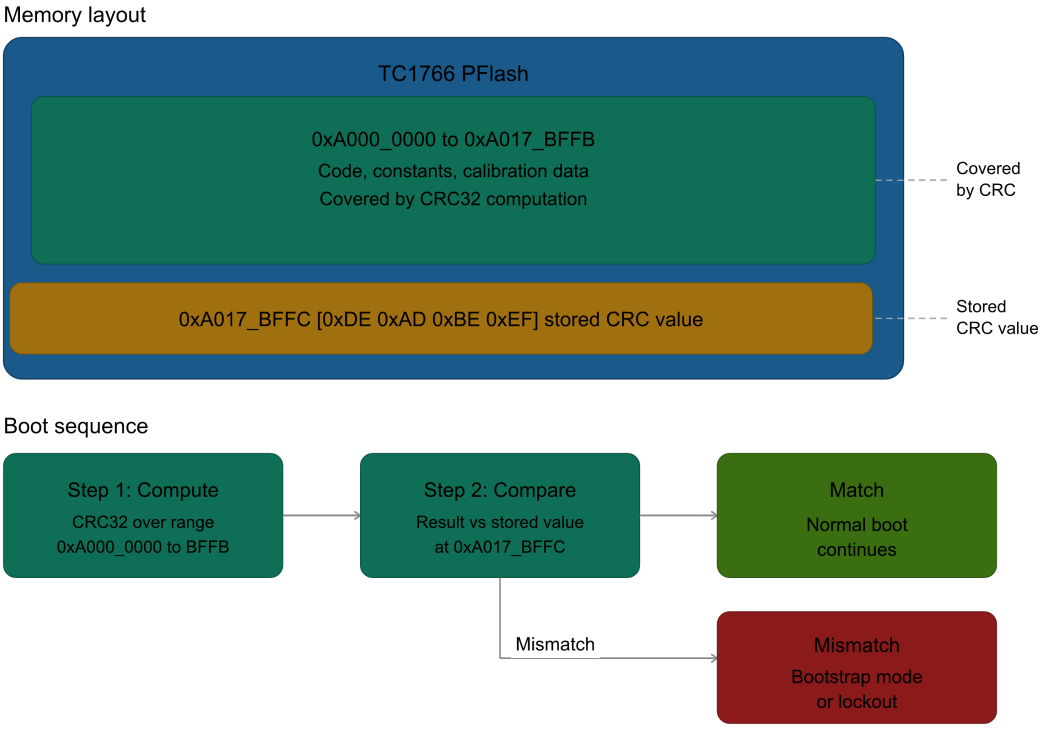


Figure I.11: TC1766 Boot-Time CRC Integrity Check Flow

Important: The Brick Problem: Patching a single byte in the firmware and reflashing it without updating the CRC will cause the ECU to detect a mismatch at next boot. Depending on the bootloader implementation, the unit may enter a locked recovery mode or become permanently inoperable. Always locate and update every checksum region before reflashing.

Multiple CRC Regions

Production firmware routinely defines multiple independent CRC regions:

- **Bootloader CRC** : verified by the bootloader itself; immutable after initial programming.
- **Application CRC** : verified by the bootloader before it transfers execution to the application.
- **Calibration CRC** : verified by the application at startup; covers the calibration data region separately.

Locating all CRC regions is a key analysis task. Search for the CRC-32 polynomial `0xEDB88320` (bit-reflected) or `0x04C11DB7` (normal form) in the disassembled binary. The computation routine is identified by its characteristic bit-shift and XOR loop structure.

Checksums and Byte Summation

Simpler ECUs employ byte summation rather than a full CRC. The sum of all bytes in a region is computed; the two's-complement of that sum is stored at a fixed address so that summing the entire region including the checksum byte yields zero. This scheme is straightforward to identify and to update.

I.6.2 Seed-Key Authentication (UDS Service 0x27)

The Security Access service is the primary authentication mechanism in UDS. Before permitting sensitive operations (flash write, direct memory read, or I/O control), the ECU issues a **seed** (a challenge value) and expects a correctly computed **key** (a response) in return.

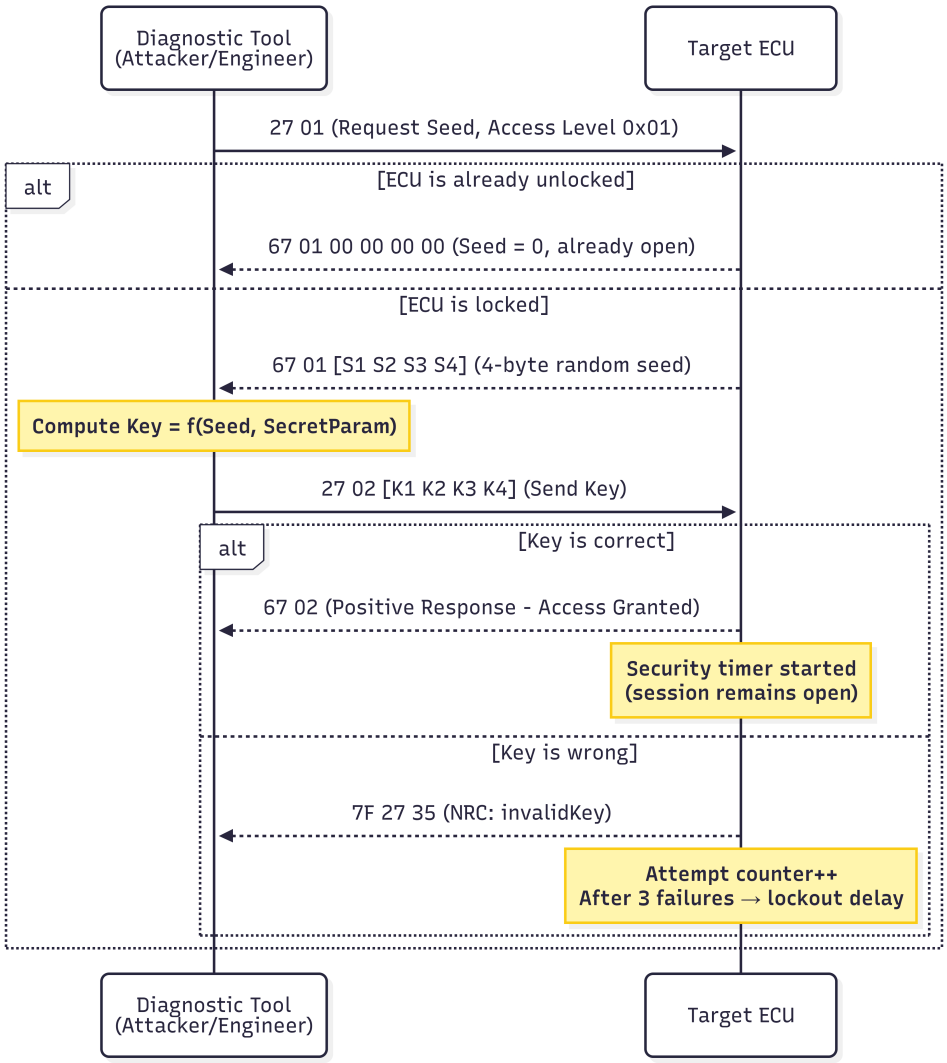


Figure I.12: UDS Seed-Key Challenge-Response Flow (Service 0x27)

II Loading your Binary

This chapter walks through every step required to correctly load the `firmware_tc1766.bin` firmware image into Ghidra and prepare the workspace before any reverse-engineering work begins. All address constants come from the TC1766 hardware reference manual and from inspecting the reset-vector header of the image itself.

II.1 Create the Project & Import the Binary

1. **File** → **New Project** → Non-Shared → give it a name.
2. **File** → **Import File** → select `firmware_tc1766.bin`.
3. In the import dialog set:
   - **Format:** `Raw Binary`
   - **Language:** `tricore:LE:32:TC176x` *(filter by "tricore")*
4. Click **Options**:
   - **Block Name:** `Pflash`
   - **Base Address:** `80000000`
5. Click **OK**, double-click the binary in the project pane, then choose **Analyze Later**; you will configure the memory map before running auto-analysis.

II.2 Configure the Memory Map

Open **Window** → **Memory Map** in Ghidra.

- If the `PFLASH` (Program Flash) block already starts at `0x80000000`, no changes are needed.
- If the `PFLASH` block starts at `0x0`, click the "home" icon (top left, labeled "base address") and set the base address to `0x80000000`.

II.3 Run Auto-Analysis

Go to **Analysis** → **Auto Analyze**, leave all options at their defaults, and click **Analyze**.

This process should only take a few seconds. On a typical firmware, Ghidra will identify around 2,900 functions after analysis. For this lab binary, it is expected that Ghidra may not automatically recognize any functions; this is normal.

II.4 Set the Entry Point

Press G and navigate to `0x80000000`, select the range `0x80000000`–`0x80000048`, and press **C** to disassemble. Then, for each sub-range below, right-click and apply the indicated data type.

Table II.1: Reset-Vector Header Layout at `0x80000000`

Offset	Start	End	Value	Meaning	Type
+0x00	0x80000000	0x80000003	0x00800090	Reset / entry pointer	Ptr
+0x04	0x80000004	0x80000007	0x00004000	Size field	Dword
+0x08	0x80000008	0x8000000B	0x80004000	RAM ptr (.data start)	Ptr
+0x0C	0x8000000C	0x8000000F	0x80003FFC	Stack top	Ptr
+0x10	0x80000010	0x80000013	0x80001AD0	(Discoverable later)	Ptr
+0x14	0x80000014	0x80000017	0x80001B00	Diag buffer descriptor ptr	Ptr
+0x30	0x80000030	0x80000033	0x7C86D5D8	CRC32 of firmware	Dword
+0x34	0x80000034	0x80000037	0x00020490	CRC region size (132 752 bytes)	Dword
+0x40	0x80000040	0x80000043	0xC0FFEE11	Firmware magic #1	Dword
+0x44	0x80000044	0x80000047	0xBEEFCA11	Firmware magic #2	Dword

II.5 Verify Core Special Function Register (CSFR) Labels

Open **Window** → **Symbol Table** in Ghidra and confirm that the following labels are present:

Table II.2: Core Special Function Register Labels

Address	Label
0xF0000010	RST_REQ
0xF0000020	WDT_CON0
0xF0000024	WDT_CON1
0xF0000210	STM_TIM0

If any label is missing, navigate to the corresponding address using **G**, right-click, and select **Add Label** to create it.

Note: Register name conventions.

- `RST_REQ` = Reset Request register.
- `WDT_CON0`/`WDT_CON1` = Watchdog Timer (WDT) Control registers 0 and 1; the WDT must be serviced periodically or it resets the chip.
- `STM_TIM0` = System Timer Module (STM) free-running 32-bit counter 0;

These four labels must be present so that Ghidra's decompiler can emit readable names rather than raw addresses.

II.6 Sanity Checks

Before moving to reverse engineering, verify the following:

- `0x80000040` reads `C0FFEE11` → binary is loaded at the correct base address.
- Memory Map shows `PFLASH` size as `0x200000`
- Labels `RST_REQ`, `WDT_CON0`, and `STM_TIM0` appear in the Symbol Table.

III Reverse Engineering

You have just loaded `firmware_tc1766.bin` into Ghidra at base `0x80000000` with language `tricore:LE:32:tc176x`, and let auto-analysis finish. You have no labels, no function names worth trusting, and a 2 MB PFLASH image.

The end goal of this section is concrete: produce a Python script that, given a 4-byte seed received over CAN from the ECU, returns the 4-byte key the ECU expects back.

To do that you must locate one specific function, read its arithmetic, and recover one constant table from PFLASH.

III.1 Why Peripherals Are Your First Anchor

A blank decompilation of any embedded firmware function looks like noise:

```
*(undefined4 *)(a0 + -0x55c0)
*(int *)(a1 + -0x7eca + idx*4)
```

There is no way to recognise "this is the seed-to-key algorithm" until those numeric offsets stop being numbers and start being *names*.

Two things make those offsets readable:

- **Absolute peripheral addresses:** The TC1766 datasheet (sections covering the Memory Map and On-Chip Peripherals) fixes the address of every on-chip register. For example the System Timer's free-running 32-bit counter is at `0xF0000210`. That address never changes. If a function reads `0xF0000210`, it is reading the system timer. No further proof is needed. Keep in mind that you cannot read the values of these registers when loading a PFLASH binary into Ghidra; you are loading a snapshot of the storage chip. The address of a register is a location inside the CPU itself, not the storage.
- **The two small-data anchor registers, `A0` and `A1`:** TriCore code does not load global variables by absolute address; it computes them as `[A0 +/- disp]` (writable globals) or `[A1 +/- disp]` (read-only constants). `A0` and `A1` are loaded once during reset, then used for the rest of the program's life. If you do not know what numeric value `A1` holds, every `[a1] - ...` reference in the listing is unreadable.

III.1.1 CSFRs

Definition

Core Special Function Registers, unlike standard registers in some other architectures that are invisible to the memory map, are mapped by TriCore to specific physical addresses so that hardware and debuggers can interact with them directly.

The CSFRs You Will Need

From the datasheet, the peripheral addresses below are the only ones needed throughout this section.

Table III.1: Required CSFR Labels for This Analysis

Address	Name	Function	Reference
`0xF0000010`	`RST_REQ`	Writing 4 here triggers a hardware reset.	Table 16-3
`0xF0000020`	`WDT_CON0`	Watchdog control 0. Software writes a password sequence here before forcing a reset.	Table 16-3
`0xF0000024`	`WDT_CON1`	Watchdog control 1, read together with CON0.	Table 16-3
`0xF0000210`	`STM_TIM0`	A 32-bit counter incrementing every system tick. Used as a cheap source of entropy.	Table 16-5
`0xF0004000` to `0xF0007FFF`	`CAN`	The MultiCAN module's register window. Resolve exact offsets from the TC1766 user manual.	Table 8-3

Action: Open *Window* > *Memory Map* in Ghidra and verify that the CAN segment appears as `F0004000` to `F0007FFF`.

Two observations to fix in mind:

1. The two WDT (Watchdog) registers and `RST_REQ` always appear together. Any function that touches all three is a hardware reset trigger.

2. `STM_TIM0` is the only 32-bit, monotonically-increasing register on the chip with no semantic side effect. Whenever firmware needs randomness without a true RNG, it reads `STM_TIM0`. Remember this; it may appear inside the seed generator.

III.1.2 The Constant-Data Anchor

While it is theoretically possible to spend a long time mapping every instruction and reconstructing the entire binary, that brute-force approach is not feasible here. The objective is surgical: isolate the seed-key algorithm without getting bogged down in the rest of the firmware.

The Architecture: Finding the Shortcut

Every TriCore-based ECU begins execution via the Boot Mode Header (BMH). This is a fixed-address data structure that tells the processor exactly where to start after a reset. The firmware header uses `C0FFEE11/BEEFCA11` magic (differs intentionally from production firmware).

Table III.2: Boot Mode Header (BMH) Layout at `0x80000000`

Offset	Address	Value	Meaning
`+0x00`	`0x80000000`	`0x00800090`	Header word (encoded, see note)
`+0x04`	`0x80000004`	`0x00004000`	Size field
`+0x08`	`0x80000008`	`0x80004000`	RAM ptr (.data start)
`+0x0C`	`0x8000000C`	`0x80003FFC`	Stack top
`+0x10`	`0x80000010`	`0x80001AD0`	Service dispatch table ptr
`+0x14`	`0x80000014`	`0x80001B00`	Diag buffer descriptor ptr
`+0x30`	`0x80000030`	`0x7C86D5D8`	CRC32 of firmware
`+0x34`	`0x80000034`	`0x00020490`	CRC region size (132,752 bytes)
`+0x40`	`0x80000040`	`0xC0FFEE11`	Firmware magic #1
`+0x44`	`0x80000044`	`0xBEEFCA11`	Firmware magic #2

While the BMH is a vital landmark, it is often just a rabbit hole for our specific goals. The first word `0x00800090` is not a directly-usable PFLASH address (PFLASH starts at

`0x80000000`), so it is either an encoded entry pointer, a header-type tag, or Firmware-specific descriptor whose interpretation requires the FBL bootloader code; we do not need to resolve it for the seed/key work. To find the seed-key logic more efficiently, focus on how TriCore handles data.

Leveraging the Small Data Area (SDA)

TriCore uses two global address registers (`A0`, `A1`) as Small Data Area (SDA) base pointers. They are set once at startup and held for the lifetime of execution. All global array/struct accesses use `[a1] +/- offset` or `[a0] +/- offset`. The seed-to-key path does not touch writable globals through `A0`, so `A0` can be ignored for now. We must recover `A1`.

Look for a pair of consecutive instructions of the following shape, somewhere in PFLASH code:

```
movh.a a1, #<imm16>          ; loads (imm16 << 16) into A1
lea    a1, [a1] <signed16>   ; adds a signed displacement
```

This is the canonical TriCore idiom for loading any 32-bit constant into an address register: the high 16 bits via `movh.a`, then a `lea` with the signed 16-bit low displacement.

The pair lives in the C-runtime startup, not the reset vector. TriCore firmware typically jumps from `_RESET` into a runtime-init routine that the linker places elsewhere in PFLASH (on this image, around `0x80100000+`).

From Mnemonic to Bytes: Encoding `movh.a a1`

A *Program Text* search only inspects bytes Ghidra has *already* decoded into the listing; anything still sitting as raw data is invisible to it, and on a fresh image the SDA setup may well be sitting in a not-yet-disassembled region. A raw **memory** (byte) search has no such blind spot, but it needs the instruction expressed as bytes. Here is how we translate the mnemonic into a searchable hex pattern.

`movh.a` uses the TriCore **RLC** instruction format: an 8-bit primary opcode, a 4-bit (unused) source slot, a 16-bit constant, and a 4-bit destination address register. For `movh.a a1, #<imm16>`, where the immediate is whatever the firmware happens to load, and is exactly the value we are trying to discover, the fields are:

Table III.3: RLC field breakdown of `movh.a a1, #<imm16>`

Bits	Width	Field	Value	Meaning
`31:28`	4	`a[c]`	`0x1`	destination register = `A1` *(pinned)*
`27:12`	16	`const16`	*(unknown)*	the immediate : left free
`11:8`	4	—	`0x0`	unused by `movh.a`
`7:0`	8	`op1`	`0x91`	`MOVH.A` opcode *(pinned)*

Only two fields are fixed no matter what value is loaded: the opcode `op1 = 0x91` and the destination nibble `a[c] = 0x1` (`A1`). The 16-bit `const16` is unknown. TriCore is **little-endian**, so the 32-bit instruction word is stored least-significant-byte first; laying the fixed fields over that byte order and **wildcarding** every nibble that belongs to `const16` (written as `.`) gives:

Table III.4: Building the wildcard search mask (`.` = wildcard nibble)

Byte	Holds	Search
byte 0	`op1` - fixed opcode	`91`
byte 1	high nibble = `const16[3:0]`; low nibble = unused (`0`)	`.0`
byte 2	`const16[11:4]` - all immediate	`..`
byte 3	high nibble = `a[c]` (`1` = A1); low nibble = `const16[15:12]`	`1.`

The resulting pattern is `91 .0 .. 1.`: opcode and destination register pinned, the immediate left free. We do *not* yet know the immediate, recovering it is the whole point of the next step.

Action: In Ghidra, open *Search > Memory*, set *Format* to **Hex**, and search for the masked pattern `91 .0 .. 1.` (each `.` is a wildcard nibble). Limit the search to the PFLASH region. This matches `movh.a a1, #<any imm16>` regardless of the immediate, and, unlike a *Program Text* search, it also catches the sequence in regions Ghidra has not yet disassembled.

Separating the Anchor from the Coincidences

The pattern is deliberately loose - only ten bits are truly pinned - so it returns **many** hits across PFLASH. A fair number are not instructions at all: they are constant tables or key material whose bytes happen to align with the `movh.a a1` encoding. You must therefore walk the results one at a time: at each address select the surrounding bytes, press `D` to disassemble that range, and apply the test below. Two representative hits make the decision concrete:

Table III.5: Triaging two `movh.a a1` hits

Property	Hit @ 0x80166506	Hit @ 0x80106a2e
Instruction	`movh.a a1,#0x431b`	`movh.a a1,#0x8003`
`A1` high half	`0x431b0000`	`0x80030000`
In firmware map?	No	Yes
Next instruction	`8c 2c 80 8c` : not a `lea`	`d9 11 b8 8d` = `lea a1,[a1]-0x25C8`
Builds a base?	No	Yes → `0x8002DA38`
Context	high-entropy data blob	C-runtime startup
Verdict	**Reject**	**Accept (the anchor)**

Concretely, four checks decide each candidate; any one of them is enough to reject:

1. **Is the immediate a plausible base?** `movh.a a1,#0x8003` gives `0x80030000`, squarely inside the firmware's address map. `#0x431b` gives `0x431b0000`, which lies in no mapped flash/RAM region and cannot be a global base.

2. **Is it immediately followed by `lea a1, [a1] <disp>`?** The SDA anchor is *always* a two-instruction pair - the high half via `movh.a`, the signed low half via `lea`. At `0x80106a2e` the next word is `d9 11 b8 8d` (`0xD9` = `LEA`) = `lea a1,[a1]-0x25C8`. At `0x80166506` the following bytes `8c 2c 80 8c` are not a `lea`: a lone `movh.a` never finishes building the pointer.

Only `0x80106a2e` passes. Disassembling that range reveals the canonical A1 initialisation sequence:

```
movh.a a1, #0x8003       ; A1 = 0x80030000
lea    a1, [a1] -0x25C8  ; A1 = 0x80030000 - 0x25C8 = 0x8002DA38
```

Thus, `A1 = 0x8002DA38`. Make a note of this value.

> **Note:** Every constant-table read for the rest of this guide is of the form `[a1] +/- something`. Without the value of `A1`, you cannot compute the absolute address being read. With it, every such reference becomes a single subtraction.

III.1.3 What You Have Now

- All on-chip peripheral CSFRs needed are labelled.
- The numeric value of `A1` is known: `0x8002DA38`.
- Any function performing writes to `WDT_CON0`, `WDT_CON1`, and `RST_REQ` is a reset trigger.
- Any function reading `STM_TIM0` is harvesting entropy.

These are weak fingerprints individually. The next sections turn them into a chain.

III.2 Where in PFLASH to Look

PFLASH on this image runs from `0x80000000` to `0x80180000`, approximately 2 MB. You will not read all of it. Almost every byte you touch from now on lies in one band: `0x80010000` to `0x800FFFFF`.

Table III.6: Approximate PFLASH Layout on This Firmware Image

Approximate Range	Typical Contents	How to Confirm
`0x80000000` to `0x80003FFF`	Boot block (FBL): CRC, version stamp	Sparse code; ASCII strings near the top with part numbers.
`0x80004000` to `0x8000FFFF`	Reset, startup, IVT, descriptor block	Dense small functions; ASCII strings like "BOSCH", part numbers.
`0x80010000` to `0x800FFFFF`	**Application code** (UDS, all diagnostic services)	Long runs of consecutive auto-recognised functions.
`0x80100000` to `0x80177FFF`	Calibration tables, lookup curves	Few or no functions; rows of monotone `int16_t` values.
Near end of PFLASH	CRC word(s), descriptor tail	Single-word reads from background tasks.

From here on, work only inside `0x80010000` to `0x800FFFFF`. In practice the SecurityAccess handler and the seed-to-key primitive both land inside the first ∼64 KB of this band, so the analysis stays within `0x80010000` to `0x8001FFFF`.

III.3 Why CAN RX Is Not the Strongest Anchor (and Why You Can Skip It)

UDS frames are delivered over CAN, so it is natural to consider tracing the firmware's logic from the CAN receive registers (`CAN_MODATAL/H`), through ISO-TP (ISO 15765-2) reassembly, to the UDS dispatcher and finally to the handler for Service ID `0x27` (SecurityAccess).

A top-down approach (starting at the CAN interrupt, following the data as it is copied into LDRAM (`0xD0xxxxxx`), then through the ISO-TP state machine, and finally to the dispatcher) does work and is instructive for understanding the protocol stack. However, on this firmware, it is not the most efficient path to the SecurityAccess handler.

You may wish to:

- **Verify CAN reception.** Identify a function that reads from `CAN_MODATAL/H` and writes to LDRAM. This confirms that CAN frames are processed and stored in RAM as expected. This is a useful sanity check, but not essential for the key extraction workflow.
- **Confirm ISO-TP presence.** Locate code that checks for ISO-TP frame types (`0x10` for First Frame, `0x21..0x2F` for Consecutive Frame, `0x30` for Flow Control). This confirms the existence of a transport layer, but you do not need to analyze its details.
- **Move on quickly.** Rather than following the protocol stack in detail, you can leverage stronger anchors that lead directly to the dispatcher and handler logic.

In summary: while tracing CAN RX and ISO-TP is possible and can help with orientation, it is not necessary for finding the SecurityAccess handler or the seed-to-key algorithm. The next anchor point, based on data structures rather than protocol flow, will get you there faster and with less ambiguity.

III.4 The Dispatcher Table: Strong Anchor, Found by Data Not Code

Naively, the right move is to find the service dispatcher and read which handler it calls when the SID is `0x27`. That approach lands on the SecurityAccess handler, but it does not work cleanly on this firmware for a specific reason: the dispatcher reads its handler-pointer table through the small-data anchor register `A1`, but the displacement from `A1` to the table

is larger than the 16-bit signed `lea` immediate can represent in one instruction. So the dispatch is implemented as a multi-instruction address build followed by an indirect call. Ghidra's static analyser cannot resolve indirect calls without help, so when you ask who calls function X for any per-SID handler, Ghidra answers "nobody", even though they are obviously called somewhere.

Do not fight this. There is a better anchor: the table itself.

III.4.1 What a UDS Service-Dispatch Table Looks Like

A UDS service-dispatch table is a logical data structure used by the ECU to map incoming Service IDs (SIDs) to their corresponding execution functions (handlers). Because it acts as static configuration data, it resides in the read-only constant range of the Program Flash (PFLASH).

Crucially, the exact structural layout of this table is dictated by the software stack vendor rather than the microcontroller hardware. Because tier-1 suppliers reuse these base software architectures across hundreds of vehicle makes, knowing the OEM ecosystem often reveals the exact memory pattern to hunt for in a binary dump.

Table III.7: Common Tier-1 UDS Dispatch Table Structures

Vendor / Stack	Entry Size	Layout Pattern
Bosch	8 bytes	Pointer (4B) + SID (1B) + Padding (3B)
Continental / Siemens	8 bytes	SID (1B) + Padding (3B) + Pointer (4B)
Delphi	Variable	Split Array: SIDs and pointers stored in separate memory blocks
AUTOSAR Stacks	12–20+ bytes	Pointer (4B) + SID (1B) + Session Masks + Security Levels

For the firmware binary we are currently analyzing, the architecture utilizes the Bosch-style 8-byte entry. This gives our target table an exceptionally distinctive fingerprint that can be recognized instantly in a hexadecimal view:

Table III.8: 8-Byte Dispatch Entry Structure (Target Firmware)

Offset	Size	Field	Characteristics
`+0x00`	4 bytes	Function Pointer	Little-endian PFLASH address. The high byte is usually `0x80` (non-cached) or `0xA0` (cached).
`+0x04`	1 byte	Service ID (SID)	A standard UDS service identifier (e.g., `0x10, 0x11, 0x22, 0x27`).
`+0x05`	3 bytes	Padding / Flags	Typically zeroes for alignment, or sparse feature flags.

The table is terminated when the 8-byte entry pattern breaks: either by a deliberate sentinel value, an unprogrammed-flash boundary (`0xFF...`), or linker-placed guard bytes. Because the terminal marker format varies between software stacks, anchoring on it is fragile. The reliable approach for this firmware is to locate the `SID 0x27` entry directly using the wildcard search in the next section, then read the table outward from that confirmed entry.

III.4.2 Hunting for the UDS Dispatch Table

Since the style of UDS dispatch table follows a strict and predictable layout, we can locate it directly by searching memory for its structural pattern. The targeted format is an 8-byte entry consisting of a 32-bit function pointer (little-endian), followed by a 1-byte Service ID (SID) and 3 bytes of zero padding.

Assuming the function pointers reside in the cached Program Flash region, which starts with `0x80`, a generic entry looks like this in memory:

```
[ B0 ] [ B1 ] [ B2 ] [ 80 ]  [ SID ] [ 00 ] [ 00 ] [ 00 ]
[ B0 ] [ B1 ] [ B2 ] [ 80 ]  [ SID ] [ 00 ] [ 00 ] [ 00 ]
[ S  ] [ S  ] [ S  ] [ S  ]  [ S   ] [ S  ] [ S  ] [ S  ]
```

If we specifically look for the SecurityAccess service (`SID = 0x27`), the byte pattern translates to: `??  ??  ??  80 27 00 00 00`

Action: In Ghidra, go to **Search** → **Memory**, select **Hex**, and enter the wildcard pattern: `??  ??  ??  80 27 00 00 00`

This search may yield a few false positives. To isolate the genuine table, inspect the memory surrounding each match. The correct region will exhibit a repeating sequence of valid 8-byte entries (a 4-byte pointer followed by a 1-byte SID and zeros).

When you find the correct location, the memory block will clearly resemble the following:

```
; Table base: 0x8001BF48   (20 entries x 8 bytes = 160 bytes, ends at 0x8001
   BFE7)
F8 9C 01 80  10 00 00 00    ; handler 0x80019CF8, SID 0x10
C2 9E 01 80  27 00 00 00    ; handler 0x80019EC2, SID 0x27  (*)
02 9E 01 80  3E 00 00 00    ; handler 0x80019E02, SID 0x3E
A8 9D 01 80  81 00 00 00    ; handler 0x80019DA8, SID 0x81
F0 9D 01 80  82 00 00 00    ; handler 0x80019DF0, SID 0x82
0C 9E 01 80  83 00 00 00    ; handler 0x80019E0C, SID 0x83
26 A0 01 80  00 00 21 00    ; handler 0x8001A026, flag-form
0E A1 01 80  00 00 E1 00    ; handler 0x8001A10E, flag-form
68 A1 01 80  01 00 E2 00    ; handler 0x8001A168, flag-form
AE A1 01 80  01 00 F4 00    ; handler 0x8001A1AE, flag-form
; ... 10 further entries follow the same 8-byte stride ...
; Guard boundary at 0x8001BFF0 (linker pad, not part of the table)
```

There are three key confirmations that this is genuinely a UDS dispatch table:

1. **Valid Pointers**: Every handler pointer ends with `0x80` (little-endian), placing it cleanly in the executable PFLASH segment.

2. **Known SIDs**: The SID column contains identifiers standard to a diagnostic ECU: `0x10` (DiagnosticSessionControl), `0x27` (SecurityAccess), `0x3E` (TesterPresent), and `0x83` (AccessTimingParameter).

3. **Sub-tables and Flag Forms**: The lower rows do not match the exact `(handler, SID, padding)` layout. Their SID byte is `0x00` and they carry non-zero values in the trailing three bytes (e.g., `21 00`, `E1 00`, `E2 00`, `F4 00`). These likely belong to a parallel sub-table for sub-functions or feature flags. The exact format does not need to be analyzed here as it is not required to reach the SecurityAccess handler.

The second row pairs `SID 0x27` with the address `0x80019EC2`. That is your next stop.

III.5 The SecurityAccess Handler at 0x80019EC2

As previously mentioned, service ID `0x27` corresponds to SecurityAccess. It is linked to the memory address `0x80019EC2`.

> **Action:** Press `G` in Ghidra and go to address `0x80019EC2`. Then open *Window > Decompile* to see the decompiled C output for `FUN_80019ec2`. If it appears as "LAB_80019ec2", right click and choose create function.

Try to understand the decompiled C code. Variable names cleaned up and structure for readability looks like this: (you may get a different code writing but they all are similar)

```
int FUN_80019ec2(uds_msg_t *req, uds_msg_t *resp)
{
    uint8_t subfunc = *(uint8_t*)((char*)req + 0x1B);
    uint8_t tag;
    int     rc;

    if (subfunc & 1) {                                  // ----- ODD parity
    -----
        if (subfunc == 0x7F) {                          //   odd special slot
            rc  = FUN_8001b6d0((uint8_t*)resp + 0x1C, 0x7F);
            *((int8_t*)resp + 0x26) = (int8_t)rc;
            tag = 0x0C;
        } else {                                        //   RequestSeed
            rc  = FUN_8001b57e((uint8_t*)resp + 0x1C, subfunc,
                               *((uint8_t*)req + 0x1C));
            *((int8_t*)resp + 0x20) = (int8_t)rc;
            tag = 0x06;
        }
    } else {                                            // ----- EVEN parity
    -----
        if (subfunc == 0x80) {                          //   even special
    slot
            rc = FUN_8001b6d0((uint8_t*)req + 0x1C, 0x80);
        } else {                                        //   SendKey
            rc = FUN_8001b57e((uint8_t*)req + 0x1C, subfunc, 0);
        }
        *((int8_t*)resp + 0x1C) = (int8_t)rc;
        tag = 0x02;
```

```
    }

    *((uint8_t*)resp + 0x19) = tag;
    if (rc == 0x34) {                                    // internal success
        rc = 0;
        *((uint8_t*)resp + 0x1B) = subfunc;              // echo subfunc
    }
    FUN_8001971c();                                      // post-validator (
    tail)
    return rc;
}
```

Three things should jump out, each a signature of the SecurityAccess service:

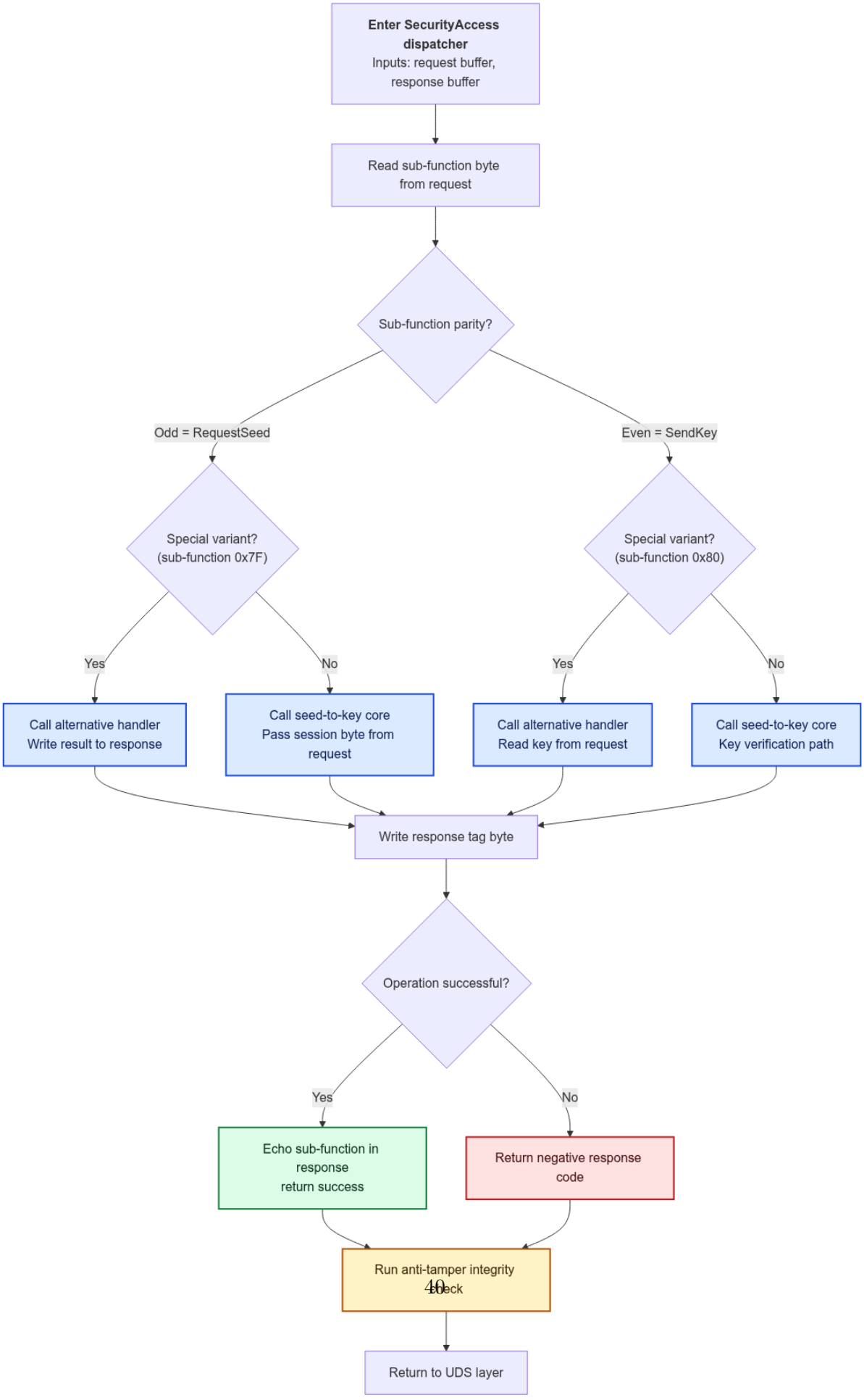
1. **Parity-first branching on a sub-function byte.** The outermost decision is `(subfunc & 1)`; inside each parity arm a second test isolates the special value (`0x7F` on the odd arm, `0x80` on the even arm). UDS service `0x27` is defined by a sub-function byte where odd values mean RequestSeed and even values mean SendKey, so a dispatcher that branches first on parity is exactly the shape you would expect.

2. **The same callee is invoked from two distinct branches with different argument shapes.** `FUN_8001B57E` is called once on the odd path with the response buffer plus a session byte taken from the request, and once on the even path with the request buffer and a zero session byte. This is a textbook fingerprint: the seed and the key flow through one shared cryptographic routine, called from RequestSeed and SendKey paths respectively. No other UDS service has this shape.

3. **The handler ends with a tail call to a post-validator** (`FUN_8001971C`) that reads magic-number globals and (on failure) the WDT / `RST_REQ` registers. Its presence is itself confirmation that you are looking at SecurityAccess.

Confirmation. `0x80019EC2` is the SecurityAccess service handler. `FUN_8001B57E`, called from both odd and even branches, is the seed-to-key primitive.

III.5.1 FUN_0x80019EC2 Diagram



III.6 Reading the Algorithm at `0x8001B57E`

> **Action:** Press `G` and navigate to `0x8001B57E`, then open *Window > Decompile*.

The decompiler will produce something close to the following (variable names cleaned up for readability):

```
// State globals (small data, addressed via TriCore a0):
//   g_sec_state_flags  @ a0 - 0x55c0   (bit 1, mask 0x02 = seed_pending)
//   g_current_sec_lvl  @ a0 - 0x558c
//   g_lfsr_state       @ a0 - 0x56fc   (rolling seed accumulator, natural
    form)
//   g_expected_key     @ a0 - 0x581c   (key for SendKey compare, natural
    form)
// Const table (read-only, via a1):
//   g_lfsr_poly_tbl[]  @ a1 - 0x7eca   (one uint32 per security level)

#define NRC_OK_INTERNAL    0x34
#define NRC_DENIED         0x33
#define NRC_INVALID_LEN    0x91
#define SEED_AWAITING_KEY  0x02

static uint32_t bit_reverse_32(uint32_t x) {
    uint32_t r = 0;
    for (int i = 0; i < 32; i++) { r = (r << 1) | (x & 1); x >>= 1; }
    return r;
}

uint32_t FUN_8001b57e(uint8_t *buf, uint32_t subfunc, uint32_t session_byte)
{
    // UDS 0x27 sub-function must be 1..0x7F (0x7F/0x80 specials are split
    off by the dispatcher)
    if ((subfunc - 1) > 0x7E) {
        g_sec_state_flags &= ~SEED_AWAITING_KEY;
        return NRC_INVALID_LEN;
    }

    if ((subfunc & 1) == 0) {
        /* ---------- SendKey (even sub-function) ---------- */
        if ((g_sec_state_flags & SEED_AWAITING_KEY) == 0)
```

```
        return NRC_DENIED;                          // no seed pending

    g_sec_state_flags &= ~SEED_AWAITING_KEY;     // one-shot
    g_current_sec_lvl  = 0;

    uint32_t received = (buf[0] << 24) | (buf[1] << 16) | (buf[2] << 8) |
buf[3];
    uint32_t diff     = received ^ g_expected_key;

    // diff must be one byte repeated four times, < 0x80
    uint8_t b0 =  diff        & 0xFF;
    uint8_t b1 = (diff >>  8) & 0xFF;
    uint8_t b2 = (diff >> 16) & 0xFF;
    uint8_t b3 = (diff >> 24) & 0xFF;
    if (b0 != b1 || b0 != b2 || b0 != b3 || b0 >= 0x80)
        return NRC_DENIED;

    g_current_sec_lvl = b0 + 1;
    return NRC_OK_INTERNAL;
}

/* ---------- RequestSeed (odd sub-function) -------- */
/* 1) Timer-nonce scramble: perturb the rolling seed using a
      poly-table slot picked by the low 6 bits of STM_TIM0.       */
uint32_t poly_natural = g_lfsr_poly_tbl[((subfunc + 1) >> 1) - 1];
uint32_t s = g_lfsr_state ^ g_lfsr_poly_tbl[STM_TIM0 & 0x3F];
g_lfsr_state = s;

/* 2) Emit the seed big-endian. This is what the tester receives;
      it is also the LFSR's starting state.                       */
buf[0] = (uint8_t)(s >> 24);
buf[1] = (uint8_t)(s >> 16);
buf[2] = (uint8_t)(s >>  8);
buf[3] = (uint8_t) s;

uint32_t rounds = session_byte + 0x23;
if (rounds > 0xFF) rounds = 0xFF;

/* 3) Bit-reversed Fibonacci LFSR:
      right-shifting state, with a bit-reversed polynomial.       */
```

```
    uint32_t poly_rev  = bit_reverse_32(poly_natural);
    uint32_t state_rev = bit_reverse_32(s);
    while (rounds--) {
        uint32_t fb = state_rev & 1u;
        state_rev = (state_rev >> 1)
                  ^ ((uint32_t)(-(int32_t)fb) & poly_rev);  // branchless
    mask
    }
    uint32_t key = bit_reverse_32(state_rev);    // back to natural form

    g_expected_key     = key;
    g_sec_state_flags |= SEED_AWAITING_KEY;
    return NRC_OK_INTERNAL;
}
```

Note: Why three bit-reversals around a right-shift loop?

A 32-bit LFSR (Linear Feedback Shift Register) computes its next state by XOR-ing selected bits of the current state with a fixed polynomial. Two equivalent forms exist:

Galois form (left-shifting, MSB feedback): `state = (state « 1) ^ (msb ? poly : 0)`

Fibonacci form (right-shifting, LSB feedback): `state_rev = (state_rev » 1) ^ (-fb & poly_rev)`

The two are *duals*: for any (seed, poly, rounds) triple they produce the same output, provided both the state and the polynomial are simultaneously bit-reversed. The proof reduces to a single identity:

$$\texttt{bit\_reverse}((s \ll 1)\ \&\ \texttt{0xFFFFFFFF}) \;=\; \texttt{bit\_reverse}(s) \gg 1$$

The handler exploits this duality in four steps:

1. Bit-reverse the polynomial into the reversed coordinate system.
2. Bit-reverse the seed into the same coordinate system.
3. Run the right-shift LFSR loop (Fibonacci form).
4. Bit-reverse the result back to the natural orientation the tester expects on the wire.

III.6.1 FUN_8001B57E Diagram

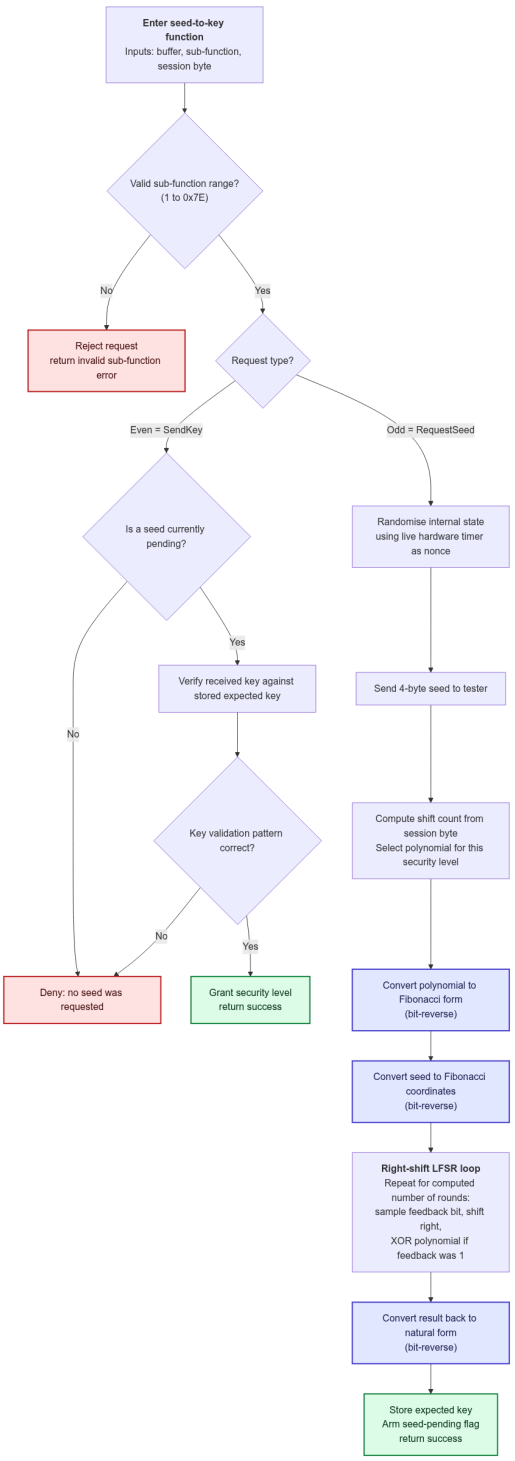


Figure III.2: `FUN_8001B57E`: bit-reversed Fibonacci seed-to-key. The three indigo nodes are the bit-reverse conversions that surround the right-shift LFSR loop; everything outside them (timer-nonce scramble, seed emit, key store, flag set) is unchanged from the natural-form algorithm.

III.6.2 The Shape of the Protocol: Odd vs. Even Sub-function

UDS Service `0x27` is a two-step protocol. The tester first sends `27 <odd_sub_function>` (RequestSeed) and the ECU replies with a four-byte seed. The tester then sends `27 <even_sub_function> <key bytes>` (SendKey) and the ECU compares the key against an internally computed expected value.

The handler operates on four globals in the small-data area, all addressed as fixed displacements from the TriCore `a0` base register: `g_sec_state_flags` at `a0-0x55c0`, `g_current_sec_lvl` at `a0-0x558c`, `g_lfsr_state` at `a0-0x56fc`, and `g_expected_key` at `a0-0x581c`. They are not fields of a struct passed into the function; the handler's parameters are just the message buffer, the sub-function byte, and a session byte taken from the request payload.

III.6.3 The Seed Generation: What the Tester Actually Receives

On the odd path, these lines are the ones that matter:

```
uint32_t s = g_lfsr_state ^ g_lfsr_poly_tbl[STM_TIM0 & 0x3F];
g_lfsr_state = s;

buf[0] = (uint8_t)(s >> 24);
buf[1] = (uint8_t)(s >> 16);
buf[2] = (uint8_t)(s >> 8);
buf[3] = (uint8_t) s;
```

The ECU reads the free-running `STM_TIM0` hardware counter and takes its low six bits as a *timing-derived selector*, not cryptographic entropy, just a value that varies between requests issued at different moments. Those six bits select one of 64 word positions starting at the table base, and that word is XORed into the persistent `g_lfsr_state` accumulator. The result is written big-endian into `buf` (this is the seed the tester receives), and `g_lfsr_state` is updated in place so the next RequestSeed call starts from a different point.

> **Note: The timer-nonce dead zone.** The polynomial table has 30 defined slots, three of which (indices 21 to 23) carry the placeholder `0xFFFFFFFF`; everything from index 30 onwards is unprogrammed flash, also `0xFFFFFFFF`. Of the 64 word positions reachable by `STM_TIM0 & 0x3F`, only 27 hold a real polynomial; the other 37 (58%) hold `0xFFFFFFFF`. Whenever the timer-nonce lands on one of those, the XOR step deterministically *inverts*

the rolling seed instead of scrambling it. The effective entropy injected per request is therefore closer to $\log_2(28) \approx 4.8$ bits than the nominal six.

Note: The seed the tester receives is also the LFSR's starting state. The tester does not need to know `STM_TIM0` or anything about the scrambling step. It only needs to take the four bytes it received and feed them directly into the LFSR loop. The write to `buf` happens *before* the LFSR loop runs, so the buffered seed is exactly the pre-roll value.

III.6.4 The LFSR Loop

```
poly      = g_lfsr_poly_tbl[((subfunc + 1) / 2) - 1];   // natural form
poly_rev  = bit_reverse_32(poly);
state_rev = bit_reverse_32(seed);

for (int i = rounds; i > 0; i--) {
    uint32_t fb = state_rev & 1u;                        // LSB instead of MSB
    state_rev   = (state_rev >> 1)                       // right-shift
                ^ ((uint32_t)(-(int32_t)fb) & poly_rev);// branchless mask
}

key = bit_reverse_32(state_rev);                         // back to natural
    form
```

This is a 32-bit Fibonacci LFSR step operating in bit-reversed coordinates. Test the *LSB* of the current state, shift the state *right* by one, and if the LSB was set, XOR with the *bit-reversed* polynomial. The conditional XOR is implemented branchlessly as `(-fb) & poly_rev`: the expression `-fb` produces `0` when `fb` is `0` and `0xFFFFFFFF` when `fb` is `1`, so ANDing with `poly_rev` selects either zero or the whole polynomial.

The reason the loop is written this way instead of the more familiar Galois left-shift form is purely about hiding the algorithm's shape from a casual reader: the right-shifting state and the extra bit-reverse calls make the function look unusual on first inspection, but the math is unchanged. The note box above the listing shows the duality.

The polynomial selected for a given access level is `g_lfsr_poly_tbl[((subfunc + 1) / 2) - 1]`, evaluated only on the RequestSeed (odd) path. RequestSeed `0x01` and its paired SendKey `0x02` both belong to access level 1 and share polynomial index 0; the pair `(0x03, 0x04)` corresponds to index 1; and so on up to the largest pair the seed/key core ever sees, `(0x7D, 0x7E)` at index 62. Sub-function `0x7F` (which would map to index 63) does not reach this code: the dispatcher `FUN_0x80019EC2` intercepts `0x7F` on its odd-parity arm

and routes it to `FUN_8001B6D0` instead. The even path does not recompute the polynomial; it simply compares against the `g_expected_key` value that the matching odd request already wrote. This is how the ECU gives each access level its own key-derivation function without a separate routine per level.

> **Note:** The same `g_lfsr_poly_tbl` is read in two distinct roles. Positions indexed by `STM_TIM0 & 0x3F` act as the timer-nonce scrambling pool on every RequestSeed; the entry indexed by `((subfunc + 1) / 2) - 1` is the LFSR feedback polynomial for that level. The two roles overlap in the same 64-word window but the timer-nonce role sees the `0xFFFFFFFF` placeholders and unprogrammed flash entries as well, while the level-poly role only reaches indices that correspond to defined access levels.

The default round count is `session_byte + 35`, where `session_byte` is the first byte of the RequestSeed payload (the dispatcher reads `request[0x1C]` and passes it in as the third parameter). A standard tester sends nothing extra, so `session_byte = 0` and the LFSR runs 35 rounds. The clamp at `0xFF` caps a malicious or oversized value and is irrelevant for real testers.

III.6.5 The Verification Check on the Even Path

```
uint32_t result = tester_key ^ g_expected_key;

if (((result & 0xFFFF) != (result >> 16)) ||
    ((result & 0xFF)   != (result >> 24)) ||
    ((result & 0xFF)   >  0x7F))
    return NRC_DENIED;

g_current_sec_lvl = (uint8_t)result + 1;
```

The three conditions together require `result` to be `0xVVVVVVVV`, where every byte is the same value `V`, and `V <= 0x7F`. The simplest passing case is `V = 0`, i.e. `result = 0`, i.e. **send `g_expected_key` directly**. This is what every real tester does.

When verification passes, `g_current_sec_lvl` is set to `V + 1`. For the standard case (`V = 0`) this equals `1`, which is the ECU's internal signal that the session is unlocked at level 1. The choice of `V` is independent of the sub-function: a tester that holds `g_expected_key` can in principle grant itself any level in `1..0x80` by picking the corresponding `V`. Access control is therefore enforced not by the granted-level byte but by the polynomial selection upstream; a tester that does not know `g_lfsr_poly_tbl[i]` cannot produce the expected key for level `i+1` in the first place.

III.6.6 The Seed-Pending Flag and Replay Protection

Bit 1 of `g_sec_state_flags` (mask `0x02`) is the seed-pending bit. It is set (`|= 0x02`) at the end of the odd path, and cleared (`&= ~0x02`) at two points: in the invalid-sub-function guard at the top of the handler, and at the start of the even path immediately after the "is a seed pending" check passes. The flag is consumed before any byte-pattern verification runs, so a failed key attempt does not need its own clear; the bit is already gone by the time the comparison happens.

The guard at the top of the even path enforces protocol order: control flow must have passed through RequestSeed first, or SendKey is rejected with `NRC_DENIED`. Because the bit is cleared as soon as the even path is entered, each seed is single-use: a second SendKey with the same key value will fail the pending-flag check even if the key would otherwise have been correct.

III.7 Recovering the Polynomial Table from PFLASH

You know `A1 = 0x8002DA38` from Section III.1. The table base is `A1 - 0x7ECA`:

```
Table base = 0x8002DA38 - 0x00007ECA = 0x80025B6E
```

> **Action:** Navigate to address `0x80025B6E` in Ghidra and read the data there in little-endian groups of 4 bytes. Verify that the first entries match the table below.

The bytes at `0x80025B6E`, in little-endian groups of 4, decode as:

Table III.9: Polynomial Table Contents at `0x80025B6E` (synthetic lab values).

Index	Bytes (LE)	Value (uint32)
0	`00 B9 18 A2`	`0xA218B900`
1	`9D FB 04 8F`	`0x8F04FB9D`
2	`FB 9C 52 F0`	`0xF0529CFB`
3	`F7 B3 83 AF`	`0xAF83B3F7`
4	`AF 2C C0 98`	`0x98C02CAF`
5	`4A B0 E0 E0`	`0xE0E0B04A`
6	`28 76 B4 A2`	`0xA2B47628`
7	`CD 30 76 FE`	`0xFE7630CD`
8	`CB 04 B4 BE`	`0xBEB404CB`
9	`96 C5 79 CE`	`0xCE79C596`
10	`1C 1F 4C D9`	`0xD94C1F1C`
11	`5E 91 82 F8`	`0xF882915E`
12	`48 BE DF D4`	`0xD4DFBE48`
13	`36 3D 8A F9`	`0xF98A3D36`
14	`CA 41 19 FC`	`0xFC1941CA`
15	`C4 FE ED F6`	`0xF6EDFEC4`
16	`30 BF 8A A6`	`0xA68ABF30`
17	`D9 AC 80 C4`	`0xC480ACD9`
18	`C2 39 01 A9`	`0xA90139C2`
19	`7C A4 00 EF`	`0xEF00A47C`
20	`9C 4B B4 D2`	`0xD2B44B9C`
21	`FF FF FF FF`	`0xFFFFFFFF`
22	`FF FF FF FF`	`0xFFFFFFFF`
23	`FF FF FF FF`	`0xFFFFFFFF`
24	`AE A6 FF 9F`	`0x9FFFA6AE`
25	`F6 A4 EB E5`	`0xE5EBA4F6`
26	`AB E3 E8 C4`	`0xC4E8E3AB`
27	`D0 29 C6 BE`	`0xBEC629D0`
28	`FF 5B 5E 9C` [49]	`0x9C5E5BFF`
29	`1D 37 7C E6`	`0xE67C371D`
30+	`FF FF FF FF`	*unprogrammed*

Indexes 21 to 23 contain `0xFFFFFFFF` between valid entries. That is not unprogrammed flash; it is a deliberate "no polynomial assigned for this level" marker, used to reserve future sub-functions `0x2B`, `0x2D`, and `0x2F`. The KeyGen rejects these slots with a clear error. From index 30 onwards, every word is genuinely unprogrammed flash. The table effectively has 27 usable entries out of 30 defined slots.

Polynomial selection per level (`((level + 1) » 1) - 1`):

Table III.10: Polynomial Selection by RequestSeed Level (synthetic lab values).

RequestSeed Sub-function	Polynomial Index	Polynomial
0x01	0	0xA218B900
0x03	1	0x8F04FB9D
0x05	2	0xF0529CFB
0x07	3	0xAF83B3F7
0x09	4	0x98C02CAF
0x0B	5	0xE0E0B04A

Action: One-shot sanity check — run the following command from the project root:

```
python3 firmware_src/synthetic_keygen.py --verify-bin firmware_src/
    synthetic_tc1766.bin
```

The script re-reads the 30 `uint32`s at `0x80025B6E` straight out of the binary and compares them against its own table. A clean run prints `OK`; any mismatch tells you exactly which slot drifted.

III.8 Implementing and Self-Verifying the KeyGen

Translate Section III.6 into Python. You can reproduce the firmware's actual bit-reversed Fibonacci form, or you can write the textbook Galois form; both produce the same output. The version below uses the simpler Galois form because it is what most readers will already know; the firmware's Fibonacci form is shown immediately after as a one-line drop-in replacement of the loop.

```
POLY_TABLE = [
    0xA218B900, 0x8F04FB9D, 0xF0529CFB, 0xAF83B3F7, 0x98C02CAF, 0xE0E0B04A,
    0xA2B47628, 0xFE7630CD, 0xBEB404CB, 0xCE79C596, 0xD94C1F1C, 0xF882915E,
    0xD4DFBE48, 0xF98A3D36, 0xFC1941CA, 0xF6EDFEC4, 0xA68ABF30, 0xC480ACD9,
    0xA90139C2, 0xEF00A47C, 0xD2B44B9C,
    0xFFFFFFFF, 0xFFFFFFFF, 0xFFFFFFFF,         # placeholders 21..23
    0x9FFFA6AE, 0xE5EBA4F6, 0xC4E8E3AB, 0xBEC629D0, 0x9C5E5BFF, 0xE67C371D,
]

def bit_reverse_32(x: int) -> int:
    x &= 0xFFFFFFFF
    x = ((x & 0x55555555) <<  1) | ((x >>  1) & 0x55555555)
    x = ((x & 0x33333333) <<  2) | ((x >>  2) & 0x33333333)
    x = ((x & 0x0F0F0F0F) <<  4) | ((x >>  4) & 0x0F0F0F0F)
    x = ((x & 0x00FF00FF) <<  8) | ((x >>  8) & 0x00FF00FF)
    x = ((x & 0x0000FFFF) << 16) | ((x >> 16) & 0x0000FFFF)
    return x & 0xFFFFFFFF

def compute_key(seed_be: bytes, level: int, loop_modifier: int = 0) -> bytes:
    if level % 2 == 0 or level < 1:
        raise ValueError("level must be a positive odd integer (RequestSeed
    sub-function)")
    idx = ((level + 1) // 2) - 1
    if idx >= len(POLY_TABLE):
        raise ValueError(f"level 0x{level:02X} -> index {idx} past table end"
    )
    poly = POLY_TABLE[idx]
    if poly == 0xFFFFFFFF:
        raise ValueError(f"level 0x{level:02X} (index {idx}) is a placeholder
    slot")

    state  = int.from_bytes(seed_be, "big")
```

```
    shifts = min(loop_modifier + 35, 0xFF)

    # --- Galois form (matches the algorithm; easier to read) -----------
    for _ in range(shifts):
        msb_was_set = state & 0x80000000
        state = (state << 1) & 0xFFFFFFFF
        if msb_was_set:
            state ^= poly
    return state.to_bytes(4, "big")

    # --- Or, if you want to mirror the firmware byte for byte: ---------
    # poly_rev  = bit_reverse_32(poly)
    # state_rev = bit_reverse_32(state)
    # for _ in range(shifts):
    #     fb = state_rev & 1
    #     state_rev >>= 1
    #     if fb:
    #         state_rev ^= poly_rev
    # return bit_reverse_32(state_rev).to_bytes(4, "big")
```

III.8.1 Single-Shift Hand Check

With one shift, the loop is trivial. Pick `seed = 0x80000000`, `level = 0x01` (so `poly = POLY_TABLE[0] = 0xA218B900`):

- MSB of `0x80000000` is 1.
- `state = (0x80000000 « 1) & 0xFFFFFFFF = 0x00000000`.
- XOR with poly: `0x00000000 ^ 0xA218B900 = 0xA218B900`.

Action: Trace one iteration of the loop by hand with `seed = 0x80000000` and `poly = 0xA218B900` (`level = 0x01`). The result must be `A2 18 B9 00`. (`compute_key` always runs at least `35` shifts, so this single-shift case is a hand check, not a function call.) Common bugs: omitting the `& 0xFFFFFFFF` mask, or testing the MSB *after* the shift instead of before.

III.8.2 Default-Shift Self-Check

Once the single-shift check passes, restore the default 35 shifts and run the following assertion:

```
assert compute_key(bytes.fromhex("A1B2C3D4"), 0x09) == bytes.fromhex("2360
    E67E")
```

Action: Execute the assertion above. With `seed = 0xA1B2C3D4`, `level = 0x09` (poly index 4, `0x98C02CAF`), and 35 shifts, the implementation must produce `0x2360E67E`. If the assertion fails, recheck the LFSR loop and the polynomial index formula.

Note: Two implementations, one answer. The reference KeyGen shipped with this lab (`firmware_src/synthetic_keygen.py`) implements the Fibonacci form and ships with a `--self-test` flag. It builds the Galois reference internally and runs both forms against a 1512-case cross-check matrix; a clean run prints `self-test passed: 1512 cross-equivalence checks`. If your own implementation passes the two checks above and disagrees with `synthetic_keygen.py` on any input, the disagreement is in your code, not the algorithm.

III.9 Anti-Tamper: What Tries to Stop You

You now have a working KeyGen. If you intend to do only tester-side work (send seed/key over CAN), you can skip this section. If you intend to attach a debugger, patch RAM, or modify the firmware, read carefully.

Note: The post-attempt validator, the WDT reset trigger, and the magic-number canaries are *unchanged* from the original ECU firmware; only the dispatcher and the LFSR core were re-implemented for the lab binary. Every trip-wire described below applies identically to both.

The SecurityAccess handler ends with a tail call to a post-attempt validator (`FUN_8001971C`). The decompiled body of that validator is:

```
void post_validator(void)
{
    snapshot_status();                          /* FUN_8001b558 */

    if (g_canary_1 != 0xA55AF00F                /* a0 - 0x5828 */
     || g_canary_2 != 0xC33C1881                /* a0 - 0x5530 */
     || FUN_8001a4bc() != 0)                    /* returns 0 == OK    */
        trigger_reset(5, 0x2F, 1);              /* hard reset, code A */

    if (FUN_80019c5a() != 1)                    /* returns 1 == OK    */
```

```
        trigger_reset(5, 0x2F, 0);                  /* hard reset, code B */
}
```

What we can state with certainty, from the body of `FUN_8001971C` alone:

- Two 32-bit canary words live in writable RAM (addressed via `a0`, so in the small-data area). Their expected values are `0xA55AF00F` and `0xC33C1881`. If either word does not match, the validator triggers a hard reset through the WDT/`RST_REQ` sequence in `FUN_80018E40`.
- Two further functions, `FUN_8001A4BC` and `FUN_80019C5A`, gate the same reset path. The first returns zero on success and is invoked alongside the canary tests; the second returns one on success and is invoked after the canary gate passes. Both call sites use the same reset routine with the same reason code (`5, 0x2F`) and differ only in the sub-reason byte (`1` vs `0`), which is what the post-mortem log will use to distinguish them on the next boot.

What we can *infer* but have not verified by disassembling those two functions:

- `FUN_8001A4BC` sits beside the canary comparisons and shares their fail path. The natural role for a function in that position on a Bosch ECU is a code- or data-integrity check (typically a CRC or simple sum over a fixed PFLASH range that includes the SecurityAccess code), but we have not confirmed this here. Treat the "code integrity" label as a working hypothesis.
- `FUN_80019C5A` is read as a boolean immediately after the integrity gate passes and is the last thing the validator checks. The shape is consistent with a timing or session-state check (e.g. "did the SecurityAccess exchange complete within an expected envelope, given the system timer captured in the request"), but again this is structural reasoning, not verified content. Confirming requires disassembling `FUN_80019C5A` and identifying what it reads.

In sequence, then, an attempt at SecurityAccess that reaches the validator must survive: the two canaries; the unverified-but-likely integrity check; and the unverified-but-likely timing/state check. Any failure writes the WDT password sequence and `RST_REQ = 4`. Before resetting, `FUN_80018E40` snapshots the program counter, parameters, and timer values into a fixed LDRAM region (the post-mortem crash log), readable through diagnostics on the next boot.

Table III.11: Anti-Tamper Trip-Wire Summary (*italics* indicate working hypotheses, not verified facts)

Action	Triggers a Reset?
Static analysis only (what you have done so far).	No
Run KeyGen against the live ECU as a tester.	No
Modify the seed or key over CAN.	No
Halt the CPU with JTAG inside `FUN_8001B57E`.	*Likely: timing/state check*
Patch a canary constant in RAM via debugger.	Yes: canary mismatch
Patch the algorithm in PFLASH.	*Likely: integrity check*

The intent of this design is clear from its shape: the firmware is friendly to external analysis and hostile to internal analysis. The path taken in this guide stays entirely on the external side, and the two "Likely" rows in the table above are the natural next analysis target if internal work becomes necessary.

III.10 What You Produced

If every step worked you now have:

- The address of the SecurityAccess service handler: `0x80019EC2` (parity-first dispatcher).
- The address of the seed-to-key primitive: `0x8001B57E` (bit-reversed Fibonacci LFSR).
- The address of the alt-handler that takes sub-functions `0x7F` / `0x80`: `0x8001B6D0`.
- The polynomial table base: `0x80025B6E`, with 30 defined slots read out of PFLASH, 27 of which carry real polynomials and three of which (indices 21 to 23) are `0xFFFFFFFF` placeholders for unassigned access levels.
- A Python KeyGen that turns any seed into the corresponding key for any supported security level, verified against a hand-computed single-shift case, a 35-shift self-check, and (via the reference `synthetic_keygen.py --self-test`) a 1512-case cross-equivalence test against the textbook Galois form.
- A clear understanding of which interventions would make the ECU reset itself, and therefore why the work in this guide stayed entirely on the tester side.

The path followed (peripherals, SDA1 base, PFLASH partitioning, SID-pattern-anchored data table, handler shape, algorithm body) is generic. With minor adaptation (different SID-byte pattern, different small-data register, different polynomial-table offset, different LFSR formulation), it applies to any UDS ECU on TriCore.

IV Testing and Validation

By the end of Chapter III you walked away with a Python KeyGen that turns any 4-byte seed into the corresponding 4-byte key. You exercised it twice: once by hand on a single-shift case, and once with a 35-shift assertion. Both matched. It felt finished.

It is not finished. Two passing tests prove the algorithm is correct *for those two inputs*. They say nothing about the other 2^{32} possible seeds, the five other supported security levels, the boundary cases where the shift count overflows, or the edge cases (zero, all-ones, single bit set) where LFSR implementations most often go wrong. This chapter closes that gap.

The strategy is to stop reasoning about correctness on paper and start *measuring* it. We will execute the firmware itself inside Ghidra's emulator, feed it inputs that we control, and read back the values it produces. Those values become the ground truth. The KeyGen you wrote in Chapter III is then judged against that ground truth on a corpus that is large enough to catch the bugs the self-tests cannot. By the end of this chapter you will have a one-command reproduction recipe and a verdict: 180 firmware-validated test cases, or a specific failure that pins down where your implementation diverges.

IV.1 From a Self-Test to an Oracle

A self-test compares an implementation to a value the author chose. Run it, get the same value back, declare success. The risk: if the author's mental model and the author's code share a bug, the self-test confirms the shared bug rather than the truth.

This is not a hypothetical risk. The classic LFSR-porting mistakes (testing the MSB *after* the shift instead of before, forgetting the `& 0xFFFFFFFF` mask in a language without native 32-bit integers, or an off-by-one in the polynomial-index formula) are systematic. The same author will reproduce the same mistake in the self-test as in the implementation, and the assertion still passes.

The escape from this trap is an *oracle*: an independent source of truth that was not written by the same author. In testing literature, an oracle answers the question "what should this input produce?" for any input we choose to ask about. There are three candidate oracles for the seed-to-key problem:

1. **The real ECU on a bench.** The most trustworthy answer, since it is the actual production device. But it requires hardware, a CAN interface, and patience; writing a wrong key during testing will eventually trip the firmware's permanent lockout. Not the right tool while still debugging.

2. **A second human implementation.** Have someone else read the firmware independently and write their own KeyGen. Compare the two. Doubles the work and only catches bugs the second author did not also make.

3. **The firmware itself, executed in an emulator.** Same bytes that run on the real ECU, same algorithm, but driven from a script. No hardware, no risk of bricking anything, and the answer is by definition correct because it *is* what the ECU would do.

Option 3 is what this chapter builds. The remaining sections explain how to make Ghidra's PCode (P-code is Ghidra's architecture-neutral intermediate representation; the PCode emulator executes it instruction-by-instruction without running native code) emulator behave like a self-contained "ask the firmware" black box, and how to wire that black box into a validator that the Chapter III KeyGen has to satisfy.

> **Note:** The word *oracle* appears throughout this chapter in its testing-literature sense: a thing that, given an input, returns the answer that defines correctness. It does not mean "a magical predictor"; it just means "a source of truth we trust more than the code we are testing". In our case, that source of truth is the firmware's own bytes, executed step by step.

IV.2 The Plan of Attack

Three pieces of software work together. Figure IV.1 shows the firmware-side picture: which function we will drive, which function we will skip, and which memory regions are involved.

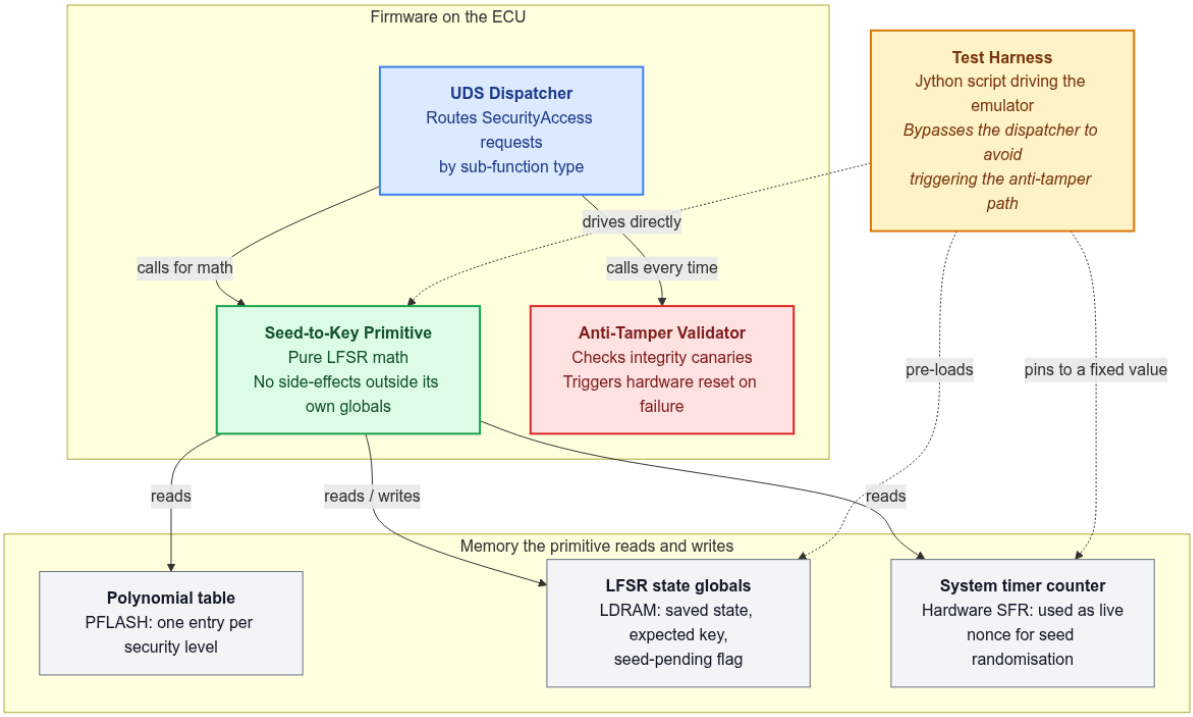


Figure IV.1: The firmware components surrounding the seed-to-key algorithm, coloured by their role in the validation strategy. Green is the function we will drive; red is the function we must avoid; blue is the dispatcher that would call both. The harness (yellow) reaches into the primitive directly, bypassing the dispatcher entirely.

In words, the recipe is:

1. Stand up a Jython script (Jython is Python 2 running on the Java Virtual Machine, the scripting language Ghidra's built-in script runner supports; the *oracle harness*) that uses Ghidra's `EmulatorHelper` (the Java API that drives Ghidra's PCode emulator programmatically) to invoke a chosen firmware function with controlled inputs.

2. For each test case, capture the two values the firmware itself emits: the seed it generated and the key it expects to receive back.

3. Persist those (seed, expected_key) pairs to a JSON (JavaScript Object Notation, a human-readable structured text format) file.

4. In a second script (the *validator*, plain CPython 3), invoke your Chapter III KeyGen as a subprocess for each captured seed, and compare its output to the firmware's expected key.

The first two pieces are new; you have not had to write or use them in any previous chapter. The next sections explain the design decisions behind each.

IV.3 Choosing the Right Function to Drive

The function the harness drives determines everything else: what registers to set up, what memory to initialise, and which side effects we have to neutralise. Pick the wrong function and the work doubles. So before writing any code, we choose carefully.

IV.3.1 The Two Candidates

Chapter III surfaced two functions that participate in SecurityAccess:

- `FUN_80019EC2` – the UDS Service `0x27` dispatcher. Receives the inbound request, decides whether it is RequestSeed or SendKey, calls the algorithm, formats the response.
- `FUN_8001B57E` – the pure seed-to-key primitive. The LFSR loop and nothing more.

At first glance the dispatcher looks like the right choice: it is the natural "entry point" a tester would see in the wild. But picking the entry point that matches the production protocol is the wrong instinct here. We are not writing a tester; we are extracting a mathematical answer from a piece of code, and the dispatcher does more than answer the math question.

IV.3.2 Why the Dispatcher Is a Trap

Look at the bottom of `FUN_80019EC2` (Section III.5). After it routes the request to the LFSR primitive, it calls `FUN_8001971C` – the anti-tamper validator catalogued in Section III.9. The anti-tamper function checks magic numbers in RAM and inspects execution timers. On a real ECU those values are established during the boot sequence: bringing up the watchdog, initialising peripherals, copying data sections from PFLASH to RAM, and so on.

In an emulator that has not booted the firmware, none of that has happened. The anti-tamper function reads the magic-number slot, finds zero, decides it has been tampered with, and writes a reset request to the WDT registers. The emulator now sits in a reset loop, and the result we wanted (the seed and the expected key) never appears.

Booting the firmware in the emulator to make the anti-tamper check pass is not impossible, but it is a project of its own: it requires modelling enough of the peripheral set to satisfy the boot code, which is a much harder problem than the one we are actually trying to solve. The easier path is to skip the dispatcher entirely.

IV.3.3 Why the LFSR Primitive Works Alone

`FUN_8001B57E` has no callees. It reads inputs from registers and from a handful of LDRAM globals, runs its loop, writes the seed to the buffer the caller provided, and stores the expected key in a fixed LDRAM slot. That is the entire side-effect surface. It does not depend on any state established during boot.

Calling `FUN_8001B57E` directly from the harness gives the same mathematical result as calling it through the dispatcher, minus the anti-tamper side trip. The two arrows leaving the dispatcher in Figure IV.1 (one arrow to the primitive, one to the validator) makes the rationale visual: we follow only one of those arrows.

> **Important:** Skipping the dispatcher works because the primitive is self-contained on the RequestSeed path. If a future firmware variant moves part of the math into the dispatcher (for example, mixing a programming-attempt counter into the seed before the LFSR runs), the same strategy would no longer suffice. The first sanity check after porting this technique to a new target is to confirm that the primitive can stand alone.

IV.4 Reading the Function's Interface from the Disassembly

Now that we know *which* function to drive, we need to know *how*. Three pieces of information are needed: what the function reads, what it writes, and where in memory those reads and writes land.

All of this is recoverable from the same disassembly you already studied in Section III.6. The difference is intent: previously we read that disassembly to understand the math; now we read it to understand the call-frame contract.

IV.4.1 What the Function Reads from Registers

The TriCore ABI (Application Binary Interface: the calling convention that specifies which registers hold function arguments, return values, and caller/callee-saved state) assigns specific registers to argument slots. In `FUN_8001B57E` the first instruction (`add d0, d4, #-0x1`) operates on `d4`, the second-argument data register; the seed-byte stores (`st.b`

`[a4]0x3, d1` and friends) target the buffer pointed to by `a4`, the first-argument address register; and `d5` feeds the loop-count adjustment. That gives us the three input arguments.

Two more registers are used as globals rather than arguments. `a1` indexes the polynomial table: `lea a2, [a1]-0x7eca` computes the base of the table at the known address `0x80025B6E`, so `a1` = `0x80025B6E` + `0x7eca` = `0x8002DA38`.

Every `[a0]-0x55c0`, `[a0]-0x56fc`, `[a0]-0x581c` elsewhere in the function indexes the LDRAM small-data block via `a0`.

Table IV.1 consolidates this.

Table IV.1: Inputs and Outputs of `FUN_8001B57E`

Register	Role	Meaning
`a4`	input	Pointer to the seed/key buffer (where the firmware writes the seed and where SendKey would read the key).
`d4`	input	Sub-function byte. Odd values request a fresh seed; the validation only exercises this path.
`d5`	input	Optional loop modifier. We leave it at `0`; the function adds `0x23` = 35 shifts internally.
`a0`	global	Small-data base for LDRAM globals (saved_state, expected_key, state flag, unlock byte).
`a1`	global	Small-data base for the polynomial table; must satisfy `a1 - 0x7eca = 0x80025B6E`.
`a11`	input	TriCore return-address register. We point it at a halt sentinel so the emulator stops when the function returns.
`a10`	input	Stack pointer. Set inside the LDRAM range, with a comfortable margin.
`d2`	output	Return code. `0x34` = ok; `0x33` = bad state; `0x91` = invalid sub-function.

IV.4.2 What the Function Reads from LDRAM

Five distinct offsets relative to `a0` appear in the disassembly. Table IV.2 translates each symbolic offset into a concrete address assuming the `a0` value chosen in Section IV.5.

Table IV.2: LDRAM Globals Used by `FUN_8001B57E`

Symbolic offset	With `a0 = 0xD000C000`	Purpose
`a0 - 0x56fc`	`0xD0006904`	*Saved state.* XORed with a timer-indexed polynomial to form the seed; persists across calls.
`a0 - 0x581c`	`0xD00067E4`	*Expected key.* The output of the LFSR; what SendKey would later compare against.
`a0 - 0x55c0`	`0xD0006A40`	State-flag bits (seed-pending, etc.).
`a0 - 0x558c`	`0xD0006A74`	Granted security level after a successful SendKey (unused on the RequestSeed path).
`a0 - 0x55e0`	`0xD0006A20`	Base for the unlock-byte store; `+0x54` aliases the byte above.

IV.4.3 What the Function Leaves Behind

Two writes survive the call and are visible to us:

- **The seed** (4 bytes, big-endian). Written byte by byte to `a4[0..3]` during the RequestSeed path. We point `a4` at a scratch buffer in LDRAM and read those four bytes back after the run.
- **The expected key** (4 bytes, little-endian `uint32`). Written by the `st.w [a0]-0x581c, d3` instruction once the LFSR loop completes. The validator will treat this value as the oracle's verdict for the corresponding seed.

These two writes are the entire payload of the call as far as the harness is concerned. Everything else (state flags, the saved-state mutation, and `d2` status) is bookkeeping we read for diagnostics.

IV.5 Designing the Harness

We now have the function's interface in hand. The harness has to build that interface around the emulator: set up the registers, plant initial values in the LDRAM globals, and arrange for the emulator to stop when the function returns. Figure IV.2 traces the resulting flow.

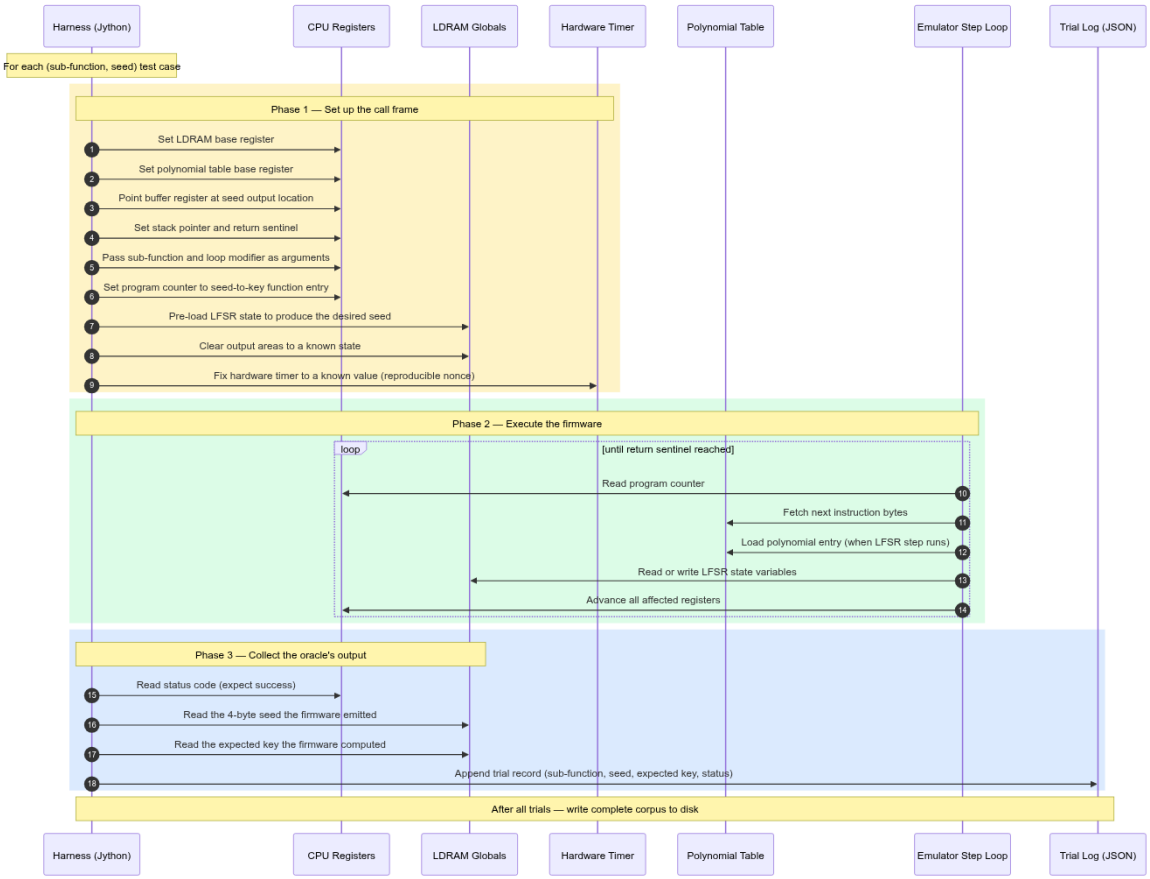


Figure IV.2: The harness executes three phases per trial: configure the call frame (yellow), step the firmware until the sentinel return address (green), and read back the two values the function produced (blue). The trial is then appended to the JSON log, which the validator consumes in the next stage.

The subsections below justify the four numeric choices the harness has to make: where to point `a0`, what sentinel value to use for `a11`, where to place the stack, and what to initialise in LDRAM before the call.

IV.5.1 Choosing `a0` So Every Reference Lands Inside LDRAM

The function reads and writes five offsets relative to `a0`. The emulator will throw an unmapped-memory fault if any of them lands outside the LDRAM block. So `a0` cannot be

arbitrary; it has to be picked so the *range* of accessed addresses stays inside the available LDRAM.

LDRAM on the TC1766 is the 56 kB region `0xD0000000`–`0xD000DFFF`. The function's offsets relative to `a0` range from `-0x581c` (most negative, so the lowest address accessed) to `-0x558c` (least negative, so the highest address accessed). The two constraints we need are:

$$\texttt{a0} - \texttt{0x581c} \geq \texttt{0xD0000000} \iff \texttt{a0} \geq \texttt{0xD000581C},$$
$$\texttt{a0} - \texttt{0x558c} \leq \texttt{0xD000DFFF} \iff \texttt{a0} \leq \texttt{0xD001358B}.$$

Any value in [`0xD000581C`, `0xD001358B`] works. We pick `a0 = 0xD000C000`, comfortably inside the range and far enough from both edges that an off-by-one in the offsets would not silently corrupt memory outside LDRAM. The concrete addresses produced by this choice are the ones in Table IV.2; all five live in the `0xD00067E4`–`0xD0006A74` band.

> **Note:** Nothing in the rest of the harness depends on the exact value of `a0`. Any value in the admissible range works the same way; the harness reads the LDRAM globals back using the same `a0 - offset` formula it used to populate them, so the chosen value cancels out.

IV.5.2 The Sentinel Return Address and the Stack

When the function executes its `ret` instruction, the TriCore CPU jumps to whatever address sits in `a11`. We want the emulator to stop there. Two requirements pin down the choice:

- The address must be *even* (TriCore PCs are two-byte-aligned; an odd PC is a hardware fault).
- It must not coincide with any real instruction in the binary, so we can detect "the function returned" by comparing the current PC to that value.

`0xDEADBEEE` satisfies both: even, easily recognised, and far above any real PFLASH or LDRAM address. The harness's step loop then reads `pc` after every instruction; when it equals the sentinel, the loop exits.

The stack pointer (`a10`) is set to `0xD000B000`. The function itself is a leaf (it makes no calls), so it does not need much stack. The chosen value leaves plenty of headroom both

below (for any local-variable allocations) and above (for the LDRAM globals we care about, which sit around `0xD000690X` to `0xD0006A7X`).

IV.5.3 Initialising LDRAM Before the Call

LDRAM is uninitialised in the emulator. If we left the globals at whatever bytes happened to be there, the function would produce non-deterministic output: the seed depends on `saved_state`, which is one of those globals. So the harness explicitly zeroes everything before each call, *then* writes the value it wants `saved_state` to hold:

```
write_u32_le(emu, SAVED_STATE_AD,  saved_state)
write_u32_le(emu, EXPECTED_KEY_AD, 0)
write_u32_le(emu, STATE_FLAG_AD,   0)
write_u32_le(emu, SEED_BUF,        0)
write_u32_le(emu, STM_TIM0,        timer_val)
```

The order matters only in that everything must be set before the function starts running. After the call, the same offsets are used to read the seed and the expected key back out.

IV.6 Controlling the Seed

The plan is now almost complete. One question remains: on a real ECU, the seed is intentionally random: it changes from call to call so that an attacker cannot replay a captured key. If the emulator faithfully reproduces that randomness, the validation becomes much harder. We would not know what seed the firmware was going to produce until after the call, and we could not write deterministic regression tests.

The good news is that the seed's "randomness" on the real ECU is shallow. Looking back at `FUN_8001B57E` in Section III.6, the seed is computed as

$$\texttt{seed} \;=\; \texttt{saved\_state} \;\oplus\; \texttt{poly\_table}[\,\texttt{STM\_TIM0} \,\&\, \texttt{0x3F}\,]\,.$$

`STM_TIM0` is the system timer at SFR address `0xF0000210`. On real silicon it ticks. In the emulator it is just a memory cell whose value we can write. `saved_state` is one of the LDRAM globals we already control.

So the seed has two "handles" the harness can grab. The technique used by the oracle script is:

1. Write a fixed value to `STM_TIM0` (the harness uses `0`). This pins the polynomial index to slot 0, which is `0xA218B900` on the synthetic lab binary.

2. For each test case where we want the firmware to emit a chosen seed s, pre-load `saved_state` with $s \oplus$ `0xA218B900`. The XOR inside the function then cancels and the firmware emits exactly s.

> **Note:** Nothing about the LFSR, the polynomial selection, or the verification check is altered by this technique. We are not faking the firmware's answer; we are only choosing which question to ask. Once the seed is chosen, the seed-to-key code path is the same one that would run on the real ECU.

The payoff: 30 different seeds per sub-function become a deterministic list of trials we can regenerate any time. That makes the corpus reproducible, diffable, and suitable for CI.

IV.7 The Reproduction Artifacts

All three files live in `PHASE::TEST/` at the root of the project tree. They are intentionally short and dependency-free, so each one can be read end to end as part of the exercise.

Table IV.3: Files in `PHASE::TEST/`

File	Runtime	Role
`keygen_oracle.py`	Jython (Ghidra)	The harness. Drives `EmulatorHelper` against `FUN_8001B57E` and dumps trial data to `/tmp/keygen_oracle_trials.json`.
`run_validation.py`	CPython 3	The validator. Replays each captured seed through the KeyGen-under-test and compares its output to the firmware-emitted key.
`synthetic_keygen.py`	CPython 3	The reference KeyGen for the lab binary (ships under `firmware_src/`). Validates your implementation against `synthetic_tc1766.bin` only.

> **Note:** The validator does not `import` the KeyGen. It runs it as a subprocess and reads its `stdout`. This is deliberate: it means the KeyGen can be written in any language as long as it speaks the command-line contract in Section IV.11.

IV.8 Step-by-Step Reproduction

The procedure assumes Chapter II is complete: `synthetic_tc1766.bin` is loaded into Ghidra at base `0x80000000` (Table II.1), and auto-analysis has finished.

IV.8.1 Install the Oracle Script

Ghidra looks for user scripts in `~/ghidra_scripts/` by default. The Jython provider is chosen based on the `@runtime Jython` directive at the top of the file, so no Java toolchain or OSGi gymnastics are needed.

> **Action:** Copy `PHASE::TEST/keygen_oracle.py` into `~/ghidra_scripts/`. Confirm Ghidra sees it by opening **Window** → **Script Manager** and looking under the *Validation* category.

IV.8.2 Run the Oracle from Ghidra

There are two equivalent ways to launch the harness; pick whichever fits your workflow.

GUI route.

1. Open `synthetic_tc1766.bin` in Ghidra's CodeBrowser.
2. **Window** → **Script Manager**.
3. Refresh the script list; `keygen_oracle.py` appears under *Validation*.
4. Double-click to run. Status messages appear in the Ghidra console.

On success the console prints one line per trial, ending with the trial count, and writes the full corpus to `/tmp/keygen_oracle_trials.json`:

```
wrote 180 trials -> /tmp/keygen_oracle_trials.json
[0x01 tim=0x00000000] ret=0x34 seed=00000000 exp_key=0x00000000 err=-
[0x01 tim=0x00000000] ret=0x34 seed=00000001 exp_key=0xDC965E00 err=-
[0x01 tim=0x00000000] ret=0x34 seed=FFFFFFFF exp_key=0xF23D5C00 err=-
...
```

The `ret=0x34` field is the firmware's status code (see Table IV.1). Any other value on any line is a red flag: it means the harness mis-configured the call, and the cross-check that follows would be measuring something other than the algorithm. Investigate before proceeding.

IV.8.3 Run the Validator

With the JSON in place, switch to a shell:

```
$ cd PHASE::TEST/
$ python3 run_validation.py
PASS sf=0x01 tim=0x00000000 seed=00000000  fw_key=00000000  keygen=00000000
PASS sf=0x01 tim=0x00000000 seed=00000001  fw_key=DC965E00  keygen=DC965E00
...
summary: 180 PASS  0 FAIL  0 ERROR  (of 180 trials)
```

A clean run prints 180 `PASS` lines, the summary, and exits with status `0`. Anything else needs interpretation, which Section IV.13 covers.

IV.9 Inside the Validator

The validator's flow is summarised in Figure IV.3. It is intentionally simple: a single loop, one subprocess call per trial, and a tally at the end.

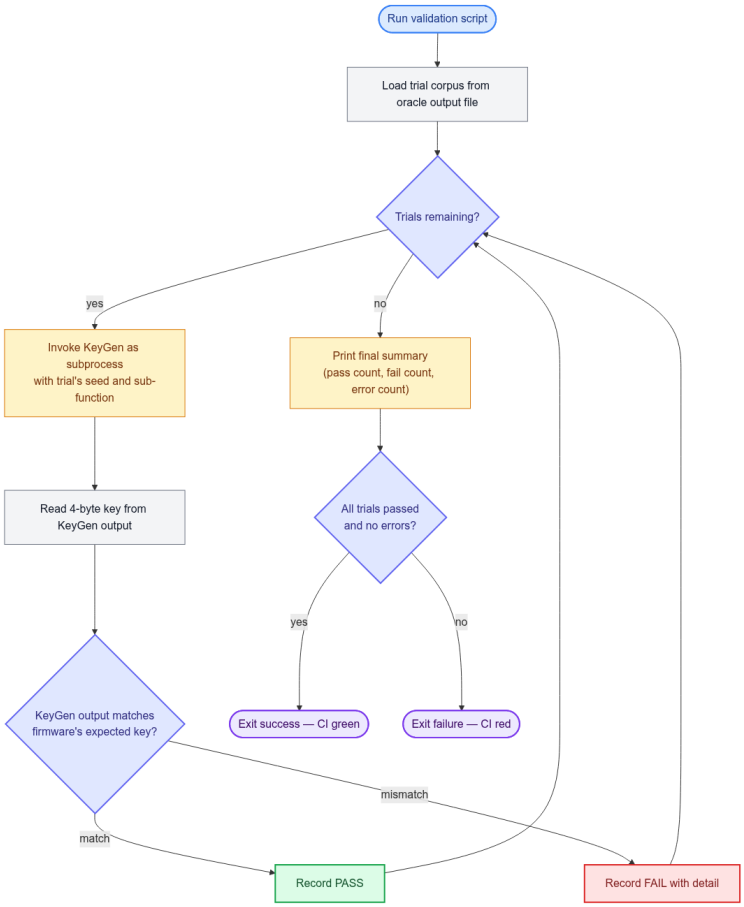


Figure IV.3: The validator loads the trial corpus produced by the oracle, invokes the KeyGen as a subprocess for each entry, and compares stdout to the firmware-emitted key. Pass/fail counts are tallied and the exit code is suitable for CI.

Two design choices in the validator are worth flagging.

Subprocess instead of import. The validator invokes the KeyGen as an external subprocess via `subprocess.check_output`, not as an imported module. Importing would be cheaper but would lock the KeyGen language to Python. A subprocess call means the KeyGen can be written in any language whose binary speaks the command-line contract: C, Rust, Go, or anything else. This matters for production work where the KeyGen ends up embedded in a diagnostic tool written in a language other than Python.

Strict string comparison. The validator compares the two key strings case-insensitively after stripping the `0x` prefix from the firmware's value. It does not parse them into integers. This catches a class of bug that a numeric comparison would silently let through: a KeyGen that prints `D0E82000` when the firmware expects `0D0E82000` (extra leading zero, hex-equal but format-wrong) fails fast, which is what we want.

IV.10 Test Corpus and Coverage

The default corpus shipped in `keygen_oracle.py` is intentionally small enough to run in a few seconds and broad enough to catch the classic LFSR-implementation bugs.

Table IV.4: Default Test Corpus

Axis	Values
Sub-functions exercised	`0x01`, `0x03`, `0x05`, `0x07`, `0x09`, `0x0B`
Seeds per sub-function	30 distinct values: zero, all-ones, single-bit boundaries, alternating-bit patterns, the polynomials themselves, arbitrary patterns.
Total trials	$6 \times 30 = 180$

The seeds are chosen to stress the parts of the LFSR loop that toy values miss: the MSB-set boundary (`0x80000000`, `0x7FFFFFFF`), behaviour at zero (where the loop must stay at zero for any shift count and any polynomial), and the polynomial constants themselves (which yield easy fixed points after the first XOR).

> **Note:** No sub-function above `0x0B` is exercised by default. Levels `0x2B`/`0x2D`/`0x2F` are the `0xFFFFFFFF` placeholder slots flagged in Section III.7; the KeyGen is expected to *reject* them, not produce a key. Levels above `0x7F` are intercepted by the dispatcher and routed to a different handler. The corpus stops at `0x0B` because that is the highest level still in production use on this firmware variant.

IV.11 Bring Your Own KeyGen

The validator does not know what algorithm the KeyGen implements; it only expects an executable that conforms to the contract below. To validate an alternative implementation, replace `synthetic_keygen.py` in `PHASE::TEST/` with the new file (keeping the name), or edit the `KEYGEN` constant near the top of `run_validation.py` to point at it.

- Invocation:

```
python3 synthetic_keygen.py <seed_hex> <subfunc_hex> --format hex
```

- Arguments: `seed_hex` is exactly 8 hex characters (4 bytes, big-endian); `subfunc_hex` is the RequestSeed sub-function as 2 hex characters.
- Output: a single line on `stdout` carrying exactly 8 hex characters (the 4-byte key, big-endian), case-insensitive.
- Exit status: `0` on success, non-zero on any error.

Important: The contract is the byte-exact *output*, not the implementation style. A KeyGen written in C, Rust, Go, or anything else is valid as long as a thin wrapper exposes the CLI above. The validator treats the KeyGen as a black box and only inspects its stdout.

IV.12 Extending the Corpus

For deeper assurance, edit the lists at the bottom of `keygen_oracle.py`.

```
subfuncs = [0x01, 0x03, 0x05, 0x07, 0x09, 0x0B]

seeds_via_saved_state = [
    0x00000000, 0xFFFFFFFF, 0x80000000, 0x7FFFFFFF,
    # ... add as many entries as desired
]
```

Three practical extensions:

- **Random sweep.** Replace `seeds_via_saved_state` with a list of, say, 10^4 random 32-bit integers produced from a fixed PRNG seed for reproducibility.
- **More sub-functions.** Append every odd byte from `0x0D` through `0x29`, skipping the placeholder slots (`0x2B`, `0x2D`, `0x2F`). Each new sub-function multiplies the trial count by the seed-list length.
- **Non-zero loop modifier.** Pass values `1..220` as the `loop_modifier` argument to `run_trial` to exercise the variable-shift-count path. The firmware caps the total at `0xFF` (255) shifts; the KeyGen must do the same.

Note: Wall-clock cost: each trial is a few thousand emulated instructions plus a small amount of Ghidra-side bookkeeping. 180 trials run in about two seconds on the reference machine; ten thousand trials run in about two minutes.

IV.13 Interpreting Failures

The validator prints one line per trial. A failing trial is unmistakable:

```
FAIL sf=0x05 tim=0x00000000 seed=12345678  fw_key=CD8AD096  keygen=CD8AD0FF
```

The two columns `fw_key` and `keygen` are the firmware's expected key and the KeyGen's output respectively. When they diverge, the question is always: *why is the KeyGen's LFSR producing a different value for this seed and this polynomial?*

Table IV.5: Common Failure Patterns

Symptom	Likely cause
All trials fail; outputs look like a bit-reversal of the expected.	MSB tested *after* the shift instead of before; or shifting right instead of left.
Trials with `seed = 0x00000000` pass; everything else off by a constant.	Polynomial-index formula wrong (missing the `((subfunc + 1) >> 1) - 1` step).
Only `sf = 0x01` passes.	Polynomial table truncated, or indexed in the wrong byte order.
Outputs differ only in the low byte.	Missing `& 0xFFFFFFFF` mask after the shift (Python-specific bug: the state silently widens beyond 32 bits).
Off by a multiple of two in the number of shifts.	Round-count formula wrong: should be `min(loop_modifier + 35, 0xFF)`, not `loop_modifier + 35` alone.

For any other pattern, capture a single failing (seed, sub-function) pair and trace it manually through the assembly in Section III.6, comparing the LFSR state after each iteration in your KeyGen to what the firmware would produce. The disagreement will be on a specific iteration; that pins down the bug.

IV.14 What Validation Buys You

Successful validation against the firmware oracle establishes three things that the static reading in Chapter III could only suggest:

1. The polynomial table read out of PFLASH was transcribed correctly. A single mistyped nibble in any of the 27 active polynomials would surface as a failure on the corresponding sub-function.

2. The shift count and the MSB test were implemented in the right order. These are the most common LFSR-porting bugs and the corpus is built to catch them.

3. The KeyGen handles edge-case seeds the same way the firmware does. The all-zero seed in particular is a useful canary because both implementations must produce an all-zero key for any number of shifts and any polynomial.

With these in hand, the KeyGen you wrote in Chapter III can be trusted for any seed the firmware might ever produce, not only the two samples used in its self-tests. That is the difference between an algorithm that “works on paper” and one that “works in production”.

A Glossary of Terms

This appendix defines every abbreviation, acronym, and technical concept used in the main text. Entries are organised thematically; within each group they are listed alphabetically.

A.1 Hardware and Architecture

ABI — *Application Binary Interface.* The calling convention that specifies which registers carry function arguments, which carry return values, and which the callee must preserve. On TriCore, data registers `d4` to `d7` carry the first four integer arguments; address registers `a4` to `a7` carry the first four pointer arguments; `d2` returns the integer result.

CSFR — *Core Special Function Register.* Memory-mapped control and status registers on the TriCore that expose hardware peripherals and CPU state. CSFRs live in the fixed range `0xF0000000` to `0xFFFFFFFF` and are accessible from software via `mfcr`/`mtcr` instructions or direct load/store to the peripheral bus.

LDRAM — *Local Data RAM.* On-chip SRAM dedicated to data storage, mapped at `0xD0000000` on the TC1766. Global variables, the stack, and the small-data area (SDA) all reside here.

PFLASH — *Program Flash.* The primary non-volatile storage for executable code on TriCore devices, mapped at `0x80000000`. The lab binary occupies the first 2 MB of this region (`0x80000000`–`0x801FFFFF`).

RST_REQ — *Reset Request Register* at CSFR address `0xF0000010`. Writing a specific pattern to this register triggers a software-initiated system reset. The anti-tamper function in the lab binary uses it to reboot the ECU on detection of an integrity violation.

SDA / SDA1 — *Small Data Area.* A TriCore compiler optimisation that places frequently-accessed global variables within a fixed ±32 KB window, allowing 16-bit signed offsets rather than full 32-bit addresses. Register `A1` holds the SDA1 base; in the lab binary `A1` is always `0x8002DA38`.

STM / STM_TIM0 — *System Timer Module.* A free-running hardware counter peripheral. `STM_TIM0` at CSFR address `0xF0000210` is the lower 32-bit word of

the counter. The seed-to-key function reads its six least-significant bits as a live nonce to vary the seed on each RequestSeed invocation.

TC1766 *Infineon TriCore TC1766.* A 32-bit automotive-grade microcontroller combining a TriCore CPU core with on-chip PFLASH, LDRAM, and automotive peripherals (CAN, LIN, FlexRay, ADC). Widely used in body-control and powertrain ECUs in the 2005 to 2015 vehicle generation.

TriCore *Infineon TriCore Architecture.* A 32-bit RISC-like CPU architecture with three orthogonal register files: 16 data registers (`D0` to `D15`), 16 address registers (`A0` to `A15`), and a set of extended 64-bit pair registers (`E0` to `E14`). Instructions are variable-width (16-bit or 32-bit); most arithmetic operates on data registers and most memory accesses use address registers.

WDT / WDT_CON0 *Watchdog Timer.* A hardware safety peripheral that resets the microcontroller if software does not service it within a configurable timeout. `WDT_CON0` at `0xF0000020` and `WDT_CON1` at `0xF0000024` are the control registers. The lab binary services the watchdog in its reset handler before jumping to the application.

A.2 Communication and Diagnostic Protocols

CAN *Controller Area Network.* A robust differential-pair serial bus standard (ISO 11898) used in automotive systems to interconnect ECUs. UDS diagnostic messages are typically encapsulated in ISO 15765-2 (CAN transport protocol, also known as ISO-TP) frames on the CAN bus.

NRC *Negative Response Code.* A one-byte status code returned by an ECU when it cannot fulfil a UDS request. Common codes used in this lab: `0x33` (Security Access Denied), `0x34` (Authentication Required: seed not yet requested), `0x35` (Invalid Key).

OBD-II *On-Board Diagnostics, version II.* A standardised diagnostic interface (SAE J1962 connector, ISO 15031 protocol) mandated in light vehicles since the mid-1990s. Provides access to emissions-related data; the same physical connector also carries OEM-specific UDS traffic.

SID *Service Identifier.* The first byte of a UDS request frame that identifies which diagnostic service is being invoked. `0x27` = SecurityAccess (used throughout this lab); `0x10` = DiagnosticSessionControl; `0x11` = ECUReset. The dispatch table in the firmware is indexed by SID.

UDS *Unified Diagnostic Services* (ISO 14229). The standard diagnostic application-layer protocol used by automotive OEMs and tier-1 suppliers for ECU programming, configuration, and access-control. A UDS session is initiated with a DiagnosticSessionControl request; SecurityAccess (service `0x27`) is then used to unlock higher privilege levels.

A.3 Security and Access Control

ECU *Electronic Control Unit.* An embedded computer that monitors and controls one or more vehicle subsystems (engine, gearbox, brakes, body, etc.). Automotive ECUs typically run on microcontrollers such as the TriCore and execute firmware stored in PFLASH.

KeyGen *Key Generator.* A software tool (here, `synthetic_keygen.py`) that, given a seed and a sub-function index, computes the expected UDS SecurityAccess key. A working KeyGen is the primary deliverable of this lab.

SecurityAccess *UDS service 0x27.* A two-step challenge-response mechanism. The tester first sends a RequestSeed message (odd sub-function: `0x01`, `0x03`, ...); the ECU responds with a 4-byte seed. The tester then sends a SendKey message (even sub-function: `0x02`, `0x04`, ...) with the computed key. A correct key grants the requested privilege level; an incorrect key returns NRC `0x35`.

Seed / Key In the UDS SecurityAccess context, the *seed* is a pseudo-random challenge value issued by the ECU, and the *key* is the expected response computed by applying the secret algorithm to that seed. Together they form a single-use challenge-response pair.

A.4 Software Tools and Environments

EmulatorHelper Ghidra's Java API for driving the PCode emulator from a script. It exposes methods to set register values, map memory regions, step

through instructions, and read results, which is the equivalent of a hardware debugger interface, but operating on PCode IR rather than native machine code.

`Ghidra` A free, open-source reverse-engineering framework developed and released by the U.S. National Security Agency (NSA). Ghidra provides disassembly, decompilation (to pseudo-C), scripting (Jython/Java), and PCode emulation. It supports over 60 processor architectures including TriCore.

`Jython` Python 2.7 compiled to run on the Java Virtual Machine. Ghidra's built-in script runner supports Jython, allowing scripts to call Ghidra Java APIs directly from Python-like syntax. The oracle harness (`keygen_oracle.py`) is written in Jython precisely because it must call `EmulatorHelper`, a Java class.

`MCP` *Model Context Protocol.* An open protocol (published by Anthropic, 2024) that defines a standard interface for AI assistants to call tools exposed by a local server. In this lab, a Ghidra MCP server translates natural-language or structured tool calls into Ghidra API invocations, enabling AI-assisted analysis of the binary.

`PCode` Ghidra's *P-code Intermediate Representation.* A register-transfer-level IR into which Ghidra translates every supported machine-code instruction. Because PCode is architecture-neutral, a single emulator engine can execute code for any supported processor. The decompiler also works from PCode, which is why it can produce pseudo-C output.

A.5 Algorithms and Cryptographic Concepts

`Fibonacci LFSR` A right-shifting LFSR formulation in which the feedback bit is the LSB (least-significant bit) of the current state XOR'd with a *bit-reversed* polynomial before being fed into the MSB. Formally: `state_rev = (state_rev >> 1) ^ (-lsb & poly_rev)`. The Fibonacci and Galois forms are mathematically dual: for the same seed, polynomial, and round count they produce the same output stream when state and polynomial are simultaneously bit-reversed. See the note box in Section III.6 for the proof.

`Galois LFSR` The canonical (left-shifting) LFSR formulation in which the feedback bit is the MSB (most-significant bit) of the current state XOR'd with a polynomial: `state = (state « 1) ^ (msb ? poly : 0)`. Named after the French mathematician Évariste Galois whose field theory underpins GF(2^n) arithmetic.

`LFSR` *Linear Feedback Shift Register.* A shift register whose input bit is a linear (XOR) function of selected bits of its previous state, called taps. The selection of taps is described by the *feedback polynomial*. LFSRs are used extensively in automotive firmware for pseudo-random seed generation and challenge-response key derivation because they are hardware-efficient and statistically well-understood.

`Polynomial (feedback polynomial)` A 32-bit constant that determines which bits of the LFSR state feed back into the shift operation. In the lab binary, 30 polynomials are stored as a lookup table at PFLASH address `0x80025B6E` (one per odd sub-function index). The choice of polynomial determines the LFSR's period and statistical properties.

A.6 File Formats and Data Notation

`Big-endian / Little-endian` Byte-ordering conventions for multi-byte integers. *Big-endian* (BE): most-significant byte first (e.g., `0x12345678` stored as `12 34 56 78`). *Little-endian* (LE): least-significant byte first (`78 56 34 12`). TriCore stores multi-byte data in little-endian order; the UDS seed and key are transmitted over the bus in big-endian order. The seed-to-key function explicitly converts between the two.

`JSON` *JavaScript Object Notation.* A lightweight, human-readable text format for serialising structured data. The oracle harness writes trial results to `/tmp/keygen_oracle_trials.json`; the validator reads this file and feeds each entry to the KeyGen.

`SHA-256` A 256-bit cryptographic hash function from the SHA-2 family. Used throughout this lab to verify that the binary has not been accidentally modified. The authoritative hash for `synthetic_tc1766.bin` is:

```
16a90e0455fba2c98fb8602e7d13449d
44523cc1f86b177bca6baa15e31af91c
```

A.7 Legal and Regulatory References

`CC BY-NC-SA 4.0` *Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International.* The licence that applies to the documentation in this lab (the report, analysis notes, and diagrams). Permits sharing and adaptation for non-commercial purposes, provided attribution is given and derivatives carry the same licence.

`CPI L.122-6-1-III` Article L.122-6-1, paragraph III of the French *Code de la Propriété Intellectuelle* (Intellectual Property Code). Permits decompilation of software for the purpose of achieving interoperability with independently developed programs, subject to conditions: the act must be performed by a licensee, only to the extent necessary, and the results may not be used for goals other than interoperability.

`EU Directive 2009/24/EC, Article 5(3)` The European Union's Software Directive, Article 5(3), permits a person authorised to use software to observe, study, or test its functioning while performing any of the acts of loading, displaying, running, or storing the software they are entitled to do. This is the EU's primary statutory basis for reverse engineering for interoperability.

`MIT Licence` A permissive open-source software licence. Permits use, copying, modification, and distribution in source and binary form, with or without modification, provided the original copyright notice is retained. Applies to the code artefacts in this lab (the generator script, keygen, and oracle harness).